



Bhartiya Vidya Bhavans Sardar

Patel Institute of Technology

Munshi Nagar – Andheri West – 400058

Name: Krish Shaw

UID: 2024300236

Branch: Comps

DIVISION: D

BATCH: D1

DATE: 30-07-2026

Experiment No.: 1

Aim : Client-Server Communication using RPC or Java RMI

Theory :

Krish Shaw Division : D
2024300236 Batch : A

DC Exp No. 1

Introduction: Distributed Computing is a computing model in which multiple computers connected through a network work together by sharing resources and exchanging information to accomplish tasks efficiently. One of the most widely used approaches in distributed computing is the Client-Server architecture, where a client sends requests for services, and a server processes these requests and returns the required results. Java provides RMI which is an implementation of RPC concept, enabling communication between applications running on different systems.

- Client-Server Architecture is a distributed computing model that separates an application into two main components:
 - Client: An application or device that requests data, services, or resources from the server.
 - Server: A system that receives client requests, processes them, performs the required operations, and sends the result back to client.

This Architecture is commonly used in web applications, banking systems, hospital management systems, and many other distributed applications.

- Advantages:
 - Centralized management of data and resources.
 - Efficient resource sharing among multiple users.

• Remote Procedure call (RPC)

Remote Procedure call (RPC) is a communication mechanism that enables a program to execute a method or procedure on a remote computer as though it were a local function call. The client sends a request to the remote server, the server performs the requested operation and result is returned to client.

- Call by value: a copy of the original value is passed to the function. Any modifications made inside the function affect only the copied value and do not change the original variable.

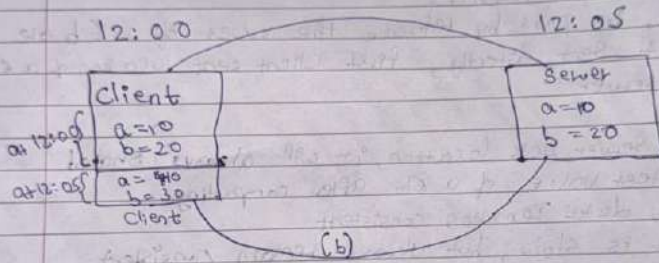
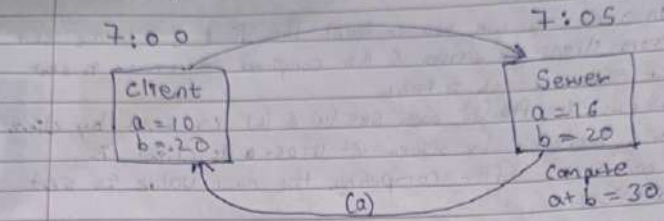
- Call by Reference: the reference of an object is passed to the function, allowing changes made within the function to directly affect the original object.

• RMI (Remote Method Invocation)

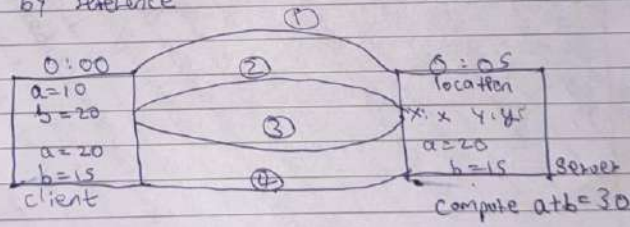
→ RMI is Java's implementation of RPC mechanism. It allows an object running in one Java VM to invoke methods on an object running in another VM over a network. This makes remote method calls appear similar to ordinary local method calls, simplifying distributed application development.

- Applications: Banking systems, online examination systems, Chat applications, Library management systems, Distributed file systems.

* Pass by Value



* Pass by reference

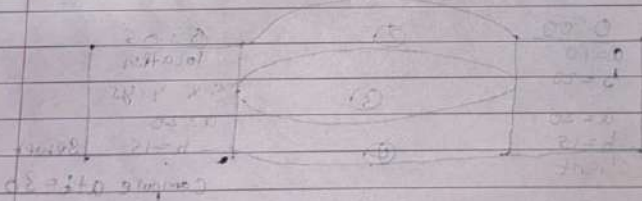


• Pass by value:

- In above: eg, we see in fig(a) the data a & b are sent from client to server & then computed at server & sent to client which is fast.
- Where in fig(b), we see (a) & (b) is sent by client and received by server at 12:05. As client is inconsistent after computing the new value is sent to client.

• Pass by reference:

- ① So, in pass by reference the values of a & b are not sent directly, first client sent location of a & b to server.
- ② If server gets location, it will always shared latest value of a & b after computing.
- ③ So, data remains consistent.
- ④ It is slow, but always remain consistent.
- ⑤ Pass by reference gives latest value present at client side.



Code :**Adder.java**

```
import java.rmi.Remote;  
import java.rmi.RemoteException;  
  
public interface Adder extends Remote {  
  
    int add(int x, int y) throws RemoteException;  
    int sub(int x, int y) throws RemoteException;  
    int mul(int x, int y) throws RemoteException;  
    double div(int x, int y) throws RemoteException;  
    int mod(int x, int y) throws RemoteException;  
}
```

AdderImpl.java

```
import java.rmi.server.UnicastRemoteObject;  
import java.rmi.RemoteException;
```

```

public class AdderImpl extends UnicastRemoteObject implements Adder {

    protected AdderImpl() throws RemoteException {
        super();
    }

    public int add(int x, int y) throws RemoteException {
        return x + y;
    }

    public int sub(int x, int y) throws RemoteException {
        return x - y;
    }

    public int mul(int x, int y) throws RemoteException {
        return x * y;
    }

    public double div(int x, int y) throws RemoteException {
        if (y == 0)
            throw new ArithmeticException("Cannot divide by zero.");
        return (double) x / y;
    }

    public int mod(int x, int y) throws RemoteException {
        if (y == 0)
            throw new ArithmeticException("Cannot take modulus by zero.");
        return x % y;
    }
}

```

RMIClient.java

```

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.util.Scanner;

public class RMIClient {

    public static void main(String[] args) {

        try {
            Registry registry = LocateRegistry.getRegistry("localhost", 1099);
            Adder stub = (Adder) registry.lookup("AdderService");

            Scanner sc = new Scanner(System.in);

```

```
int choice;

do {
    System.out.println("\n===== MENU =====");
    System.out.println("1. Addition");
    System.out.println("2. Subtraction");
    System.out.println("3. Multiplication");
    System.out.println("4. Division");
    System.out.println("5. Modulus");
    System.out.println("6. Exit");
    System.out.print("Enter your choice: ");

    choice = sc.nextInt();

    if (choice >= 1 && choice <= 5) {

        System.out.print("Enter first number: ");
        int a = sc.nextInt();

        System.out.print("Enter second number: ");
        int b = sc.nextInt();

        switch (choice) {
            case 1:
                System.out.println("Result = " + stub.add(a, b));
                break;

            case 2:
                System.out.println("Result = " + stub.sub(a, b));
                break;

            case 3:
                System.out.println("Result = " + stub.mul(a, b));
                break;

            case 4:
                System.out.println("Result = " + stub.div(a, b));
                break;

            case 5:
                System.out.println("Result = " + stub.mod(a, b));
                break;
        }
    }
}
```

```

        } else if (choice == 6) {
            System.out.println("Exiting...");
        } else {
            System.out.println("Invalid Choice!");
        }

    } while (choice != 6);

    sc.close();

} catch (Exception e) {
    System.out.println("Client Exception: " + e);
    e.printStackTrace();
}
}
}

```

RMIServer.java

```

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class RMIServer {
    public static void main(String[] args) {
        try {
            // Create the remote service instance
            Adder stub = new AdderImpl();

            // Create and start the RMI Registry on port 1099 locally
            Registry registry = LocateRegistry.createRegistry(1099);

            // Bind the stub instance to the name "AdderService"
            registry.rebind("AdderService", stub);

            System.out.println("RMI Server is running and ready...");
        } catch (Exception e) {
            System.err.println("Server exception: " + e.toString());
            e.printStackTrace();
        }
    }
}

```

Output screenshot :

```
===== MENU =====
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Modulus
6. Exit
Enter your choice: 1
Enter first number: 10
Enter second number: 20
Result = 30
```

```
===== MENU =====
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Modulus
6. Exit
Enter your choice: 2
Enter first number: 50
Enter second number: 30
Result = 20
```

```
===== MENU =====
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Modulus
6. Exit
Enter your choice: 3
Enter first number: 4
Enter second number: 5
Result = 20
```

```
===== MENU =====
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Modulus
6. Exit
Enter your choice: 4
Enter first number: 40
Enter second number: 8
Result = 5.0
```

```
===== MENU =====
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Modulus
6. Exit
Enter your choice: 5
Enter first number: 11
Enter second number: 7
Result = 4
```

```
===== MENU =====
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Modulus
6. Exit
Enter your choice: 6
Exiting...
```

Conclusion:

Through this experiment, I gained a clear understanding of client-server communication using Java Remote Method Invocation (RMI). I learned how distributed applications communicate by allowing a client to invoke methods on a remote server object as if they were local method calls. I understood the roles of the remote interface, remote object implementation, RMI Registry, server, and client in establishing communication. I also learned how to register remote objects, look up services from the registry, and handle remote exceptions during execution. Additionally, this experiment provided practical experience in developing, deploying, and testing a distributed Java application, strengthening my understanding of the basic concepts of distributed systems and remote communication over a network.

ALIVE: A Latency- and Cost-Aware Hybrid P2P-CDN Framework for Live Video Streaming

Reza Farahani¹, Member, IEEE, Ekrem Çetinkaya, Member, IEEE, Christian Timmerer², Senior Member, IEEE, Mohammad Shojafar³, Senior Member, IEEE, Mohammad Ghanbari⁴, Life Fellow, IEEE, and Hermann Hellwagner⁵, Senior Member, IEEE

Abstract—Recent years have witnessed video streaming demands evolve into one of the most popular Internet applications. With the ever-increasing personalized demands for high-definition and low-latency video streaming services, *network-assisted video streaming* schemes employing modern networking paradigms have become a promising complementary solution in the *HTTP Adaptive Streaming* (HAS) context. The emergence of such techniques addresses long-standing challenges of enhancing users' *Quality of Experience* (QoE), *end-to-end* (E2E) latency, as well as *network utilization*. However, designing a cost-effective, scalable, and flexible network-assisted video streaming architecture that supports the aforementioned requirements for live streaming services is still an open challenge. This article leverages novel networking paradigms, *i.e.*, *edge computing* and *Network Function Virtualization* (NFV), and promising video solutions, *i.e.*, HAS, *Video Super-Resolution* (SR), and *Distributed Video Transcoding* (TR), to introduce **A Latency- and cost-aware hybrid P2P-CDN framework for live video streaming (ALIVE)**. We first introduce the **ALIVE** multi-layer architecture and design an *action tree* that considers all feasible resources (*i.e.*, storage, computation, and bandwidth) provided by peers, edge, and CDN servers for serving peer requests with acceptable latency and quality. We then formulate the problem as a *Mixed Integer Linear Programming* (MILP) optimization model executed at the edge of the network. To alleviate the optimization model's high time complexity, we propose a lightweight heuristic, namely, *Greedy-Based Algorithm* (GBA). Finally, we (i) design and instantiate a large-scale cloud-based testbed including 350 HAS players, (ii) deploy **ALIVE** on it, and (iii) conduct a series of experiments to evaluate the performance of **ALIVE** in various scenarios. Experimental results indicate that **ALIVE** (i) improves the users' QoE by at least 22%, (ii) decreases incurred cost of

the streaming service provider by at least 34%, (iii) shortens clients' serving latency by at least 40%, (iv) enhances edge server energy consumption by at least 31%, and (v) reduces backhaul bandwidth usage by at least 24% compared to baseline approaches.

Index Terms—HTTP adaptive streaming (HAS), edge computing, network function virtualization (NFV), content delivery network (CDN), peer-to-peer (P2P), quality of experience (QoE), video transcoding, video super-resolution.

I. INTRODUCTION

THE EMERGENCE of modern networking (*e.g.*, 5G and 6G) technologies, the development of Over-The-Top (OTT) content services (*e.g.*, YouTube, Netflix, Facebook), and steadily growing numbers of users who prefer to stream video content over the Internet rather than using classical TV, have made video streaming the dominant traffic on the Internet. According to statistics, video streaming applications represent about 66% of the total traffic over the Internet, and live video streaming will account for more than 17% of the whole video traffic by 2023 [1]. Multifarious video delivery technologies have emerged in recent years in order to improve video streaming services. Among them, HAS-based solutions, such as *Dynamic Adaptive Streaming over HTTP* (DASH) [2] and Apple's *HTTP Live Streaming* (HLS) [3], with delivering more than 50% of video streams, have become the most prevalent technologies utilized by OTT providers for live video streaming applications [4]. In HAS-enabled systems, video sequences are split into short- and fixed-duration segments (*i.e.*, between two to ten seconds durations), and then each segment is encoded into multiple versions called *representations* (*i.e.*, resolutions or bitrate levels). Information on the segments, including their locations on media servers, *e.g.*, *Content Delivery Network* (CDN) servers or origin server, is stored within the *Media Presentation Description* (MPD). HAS players then process MPDs and consider the current network status and/or capabilities of the users' equipment (*e.g.*, playout buffer) to adaptively download appropriate representations from media servers using *Adaptive Bitrate* (ABR) algorithms, consequently improving users' QoE [5].

A. Motivation

Although employing CDN services to scale HAS delivery systems has been a step forward, the surge in video traffic and tremendous growth in demanding high-quality and low-latency

Manuscript received 17 February 2023; revised 21 July 2023 and 13 November 2023; accepted 15 November 2023. Date of publication 28 November 2023; date of current version 15 April 2024. The financial support of the Austrian Federal Ministry for Digital and Economic Affairs, the National Foundation for Research, Technology and Development, and the Christian Doppler Research Association are gratefully acknowledged. Christian Doppler Laboratory ATHENA: <https://athena.itec.aau.at/>. The associate editor coordinating the review of this article and approving it for publication was D. Rossi. (Corresponding author: Reza Farahani.)

Reza Farahani, Ekrem Çetinkaya, Christian Timmerer, and Hermann Hellwagner are with the Christian Doppler Laboratory ATHENA, Institute of Information Technology (ITEC), Alpen-Adria-Universität Klagenfurt, 9020 Klagenfurt, Austria (e-mail: Reza.Farahani@aau.at; Ekrem.Cetinkaya@aau.at; Christian.Timmerer@aau.at; Hermann.Hellwagner@aau.at).

Mohammad Shojafar is with the 5G & 6G Innovation Centre, Institute for Communication Systems, University of Surrey, GU2 7XH Guildford, U.K. (e-mail: m.shojafar@surrey.ac.uk).

Mohammad Ghanbari is with the School of Computer Science and Electronic Engineering, University of Essex, CO4 3SQ Colchester, U.K. (e-mail: ghan@essex.ac.uk).

Digital Object Identifier 10.1109/TNSM.2023.3335190

live video streams creates considerable challenges to OTT providers. For instance, increasing the number of video users in popular events like football matches can overload CDN servers; as a result, OTT services fail to deliver satisfactory quality and latency to end-users and lead to severe pressure on backhaul networks as a bottleneck of video delivery systems [6]. Several studies have shown that such problems force users to abandon live sessions quickly, consequently causing significant revenue losses and imposing extra costs to OTT companies [6]. In addition, although HAS-based delivery approaches certainly have merits over non-HAS streaming systems, most of the existing HAS-based methods are pure client-based solutions, where the players' ABR algorithms often make bitrate decisions locally, only considering buffer status and estimated available bandwidth without a global network view. Given the large-scale distributed nature of the Internet, individual nodes only observe a partial fragment of the video streaming system, which results in imperfect adaptations and poor long-tail performance [4].

The aforementioned problems and requirements of assuring the users' QoE have triggered OTT and Internet service providers (ISPs) to collaborate, upgrade their video delivery systems and introduce an emerging solution called *network-assisted video streaming* (NAVS) [4], [6]. In NAVS systems, an in-network component (e.g., edge server, in close proximity to users) with a broader view of the network is employed to assist video players in making precise adaptation decisions, hence improving users' QoE and latency [7], [8], [9], [10], [11], [12], [13]. Most edge-based NAVS solutions primarily leverage edge storage resources to reduce the streaming transmission latency. However, due to limitations of edge server resources (e.g., computation, storage, and bandwidth), such frameworks cannot efficiently improve video delivery systems during bursty situations, i.e., when the number of live requests increases. Thus, there is a need to design a cost-aware, scalable, and flexible edge-based NAVS scheme that supports low-latency and high-quality live video streaming.

Recent studies have shown that utilizing clients' capabilities within a P2P network to form hybrid P2P-CDN video delivery systems can address the aforementioned issues and offer many benefits, like alleviating network congestion, increasing streaming stability, and reducing delivery costs [5], [13], [14], [15]. Considering these advantages, many companies, e.g., *Peer5*, *Strivecast*, *Livepeer*, and *Viblast*, have been utilizing peer-assisted video streaming solutions with promising networking protocols (e.g., WebRTC) to offload CDNs and provide low-latency and high-quality services. As a result, incorporating hybrid P2P-CDN with HAS technology could be employed to optimize video streaming delivery, especially in live video scenarios.

B. Objectives

Most of the existing works on presenting peer-assisted live video streaming solutions (see Section II) do not effectively use the peers' resources to provide high-quality and low-latency services. On the other hand, edge-based NAVS

schemes overlook how to use the idle resources of clients to optimize users' QoE and network utilization. Motivated by the above-mentioned issues, the *primary* goal of this article is to design a hybrid P2P-CDN system for live video streaming applications to (i) efficiently utilize all resources (i.e., computing, storage, and bandwidth) provided by both P2P and CDN nodes, (ii) leverage modern networking paradigms, i.e., NFV [16], and edge computing to introduce virtualized tracker servers at the edge (refer to Section III-B for details), and (iii) satisfy HAS client requests with acceptable QoE and latency. Since we want to introduce a framework deployable on any hybrid P2P-CDN network, we assume that a hybrid P2P-CDN network is constructed in advance. For the sake of simplicity, we also suppose that NAT and firewall configurations have been appropriately managed and adjusted to meet the requirements of HAS requests. Therefore, we propose **A** Latency- and cost-aware hybrid P2P-CDN framework for **li**Ve video strEaming (ALIVE). Indeed, ALIVE is an edge-based hybrid P2P-CDN NAVS system that exploits the peers' idle resources to provide distributed video transcoding (TR) and super-resolution (SR) services besides video transmission. Moreover, ALIVE augments edge servers with video transcoding and partial caching functionalities in the form of *Virtualized Network Functions* (VNFs). Employing peers' SR (i.e., upscale the available lower resolution representation to reconstruct the requested representation) and peers' TR (i.e., rebuild the requested representation from the available higher bitrate representation) functionalities, besides edge caching and transcoding and CDN caching, enables ALIVE to use all provided resources efficiently for achieving the mentioned goals. Note that utilizing VNF-based techniques to form ALIVE's edge servers makes them deployable on any computing or networking appliance. The novelty of ALIVE in terms of design are *three-fold*:

- Devise virtualized edge-based tracker servers (deployable on any networking servers) to control the proposed hybrid P2P-CDN framework.
- Introduce peer TR and SR functionalities to utilize all provided P2P resources to reduce network costs, improve users' QoE, and prevent edge server overloading.
- Employ an action tree to select the best strategies (i.e., fetching, TR, or SR) to respond to the clients' requests from the best nodes (i.e., peer, edge, CDN, or origin server) with reduced latency, improved QoE, and enhanced network utilization.

C. Challenges and Contributions

In the design of the proposed framework, the following challenges must be taken into account.

- Serving clients' requests from (i) suitable locations (i.e., CDN, origin, peers, edge) in terms of the latency and QoE and (ii) utilizing appropriate policies (i.e., fetching, TR, or SR) by jointly considering network conditions, QoE objectives, and resource constraints requires a broad view of the network. Hence, designing a scheme that considers peers, CDNs, and edge servers' capabilities and limitations for decision-making is vital.

- SR and TR functions are computationally-intensive and power-hungry tasks. Therefore, selecting the best place for deploying them and employing lightweight methods, especially over peers with limited power and computational resources, is crucial to engage all resources efficiently.
- Collaborating among different business stakeholders, *e.g.*, ISPs, CDN providers, OTT companies, and P2P networks, is required for implementing ALIVE. To this aim, edge nodes are enabled to host third-party services, *e.g.*, proxy servers of OTT providers, of mobile network operators (*e.g.*, MobileEdgeX), and of CDN providers (*e.g.*, Cloudflare and Limelight).

The principal contributions of this article are as follows:

- *Architecture.* We leverage modern networking (*i.e.*, P2P, CDN, NFV, edge computing) and video (*i.e.*, HAS, TR, and SR) paradigms to design a multi-layer hybrid P2P-CDN architecture. We also introduce an action tree including all possible actions for serving clients' requests employed by *Virtual Tracker Servers* (VTSs) at the edge of a P2P-CDN network (*i.e.*, servers between peers and CDNs).
- *Problem formulation.* We formulate the problem as a Mixed Integer Linear Programming (MILP) optimization model to jointly optimize overall E2E latency and network cost while considering resource limitations.
- *Heuristic algorithm.* Due to the NP-completeness of the proposed MILP model, we propose a lightweight *Greedy*-based heuristic algorithm (GBA) method.
- *Implementation.* We design and instantiate a large-scale cloud-based testbed, including 350 HAS clients, to deploy ALIVE and analyze its performance.
- *Evaluation.* We show an extensive analysis of the ALIVE performance by conducting comprehensive experiments and comparing the results with baseline schemes. The results demonstrate that the ALIVE approaches significantly benefit OTT providers and network operators in terms of users' QoE (by at least 22%), network costs (by at least 34%), serving latency (by at least 40%), edge server energy consumption (by at least 31%), and backhaul bandwidth usage (by at least 24%) compared to traditional CDN-based and several hybrid P2P-CDN baseline methods.

D. Article Outline

The remainder of this paper is organized as follows. Section II reviews the related work and highlights the novelties of the ALIVE framework. In Sections III-A and III-B, we discuss the ALIVE problem definition and explain the proposed ALIVE multi-layer architecture, respectively. We formulate the problem as an MILP optimization model in Section III-C. Section IV explains the proposed lightweight heuristic algorithm. The evaluation setup and results are presented in Section V. Finally, we conclude the article and outline future work in Section VI.

II. RELATED WORK

This section studies CDN-based (Section II-A), P2P-based (Section II-B), and hybrid (Section II-C) network-assisted

video streaming (NAVS) solutions and highlights our work's novelties.

A. CDN-Based NAVS Solutions

With emerging HAS-based technologies, OTT companies often utilize distributed CDN servers in different geographical locations to get more reliable and cost-effective content delivery. Many works such as [17], [18], [19], [28] propose new caching strategies to improve CDN hit rates and eventually enhance video QoE. In contrast, ALIVE is independent of the caching policies and sheds light on how the requested video segments are served from existing CDNs or peers with acceptable E2E latencies and QoE. Furthermore, we demonstrate that CDN may not constantly improve users' QoE, and the results are improved via both CDN and P2P capabilities. Some works use a centralized component, *e.g.*, a *Software Defined Network* (SDN) controller or an edge server, to assist HAS players in their ABR decisions, bandwidth allocation, path selection or selecting CDNs dynamically [8], [9], [10], [11], [12], [20], [25], [29]. Conversely, we propose a hybrid P2P-CDN approach for supporting latency-sensitive services, *i.e.*, live video streaming.

Some standardization activities, *e.g.*, [30], [31], [32] try to mitigate HAS performance problems by enabling protocol messages to be exchanged among network components, *e.g.*, CDN servers. Although these systems can enhance ABR decisions, their implementation could not be straightforward, and only a few papers have followed these approaches in the traditional CDN-based fashion [13]. Zhang et al. [21] introduced a video super-resolution and caching system named VISCA, which utilizes video SR at the edge of the network and designs an edge-based ABR algorithm to improve users' QoE. However, since the output of the SR module is raw-input video, recompression is needed to transmit the reconstructed video segment. Thus, extra delay and distortion will be imposed by the system. Therefore, the best place for applying the SR technique is the P2P network, where the decoded segments will immediately play out after reconstruction by peer SR.

B. P2P-Based NAVS Solutions

Peer-assisted content delivery networks have been considered an economical alternative to traditional CDN-based approaches. In such systems, clients (so-called *peers*) act as both clients and servers and contribute their resources, *e.g.*, storage and bandwidth, to increase content availabilities for groups of peers, called *content swarms*. Classical P2P applications are network oblivious, where peers do not consider the underlying network topology when choosing their neighbors; this may lead to inefficient network resource usage [33]. The solution introduced by some prior works like [34] for establishing a collaboration between network service providers and content providers led to an Internet Engineering Task Force (IETF) standard called *Application-Layer Traffic Optimization* (ALTO) [35]. ALTO enables P2P applications to obtain abstract maps of network information, including from CDNs, to optimize network resource consumption and traffic delivery while maintaining application

TABLE I
OVERVIEW OF RELATED ARTICLES; TR := TRANSCODING; SR := SUPER-RESOLUTION

Works	Streaming Type	NAVS Type	CDN Function(s)	Edge Function(s)	Peer Function(s)	Optimization Model	Heuristic	Evaluation
[7]	×	CDN-Based	Caching	×	×	×	×	Real Testbed
[8]	×	CDN-Based	Caching	caching	×	✓	✓	Simulation
[17]	×	CDN-Based	Caching, TR	×	×	✓	✓	Simulation
[18]	×	CDN-Based	Caching	×	×	✓	✓	Trace-Driven
[19]	×	CDN-Based	Caching	Caching	×	×	×	Trace-Driven
[9]	×	CDN-Based	Caching	Caching	×	✓	×	Real Testbed
[12], [20]	×	CDN-Based	Caching	Caching, TR	×	✓	×	Real Testbed
[21]	×	CDN-Based	Caching	Caching, SR	×	×	✓	local Testbed
[22]	×	P2P-Based	Caching	×	Caching	×	×	Akamai Product
[14]	Live	Hybrid-Based	Caching	Caching	Caching	×	✓	Simulation
[23]	Live	Hybrid-Based	Caching	×	Caching	×	✓	Trace-Driven
[24]	Live	Hybrid-Based	Caching	×	Caching	×	×	Local Testbed
[25]	Live	Hybrid-Based	Caching, TR	Caching	Caching	×	✓	Simulation
[26]	Live	Hybrid-Based	Caching	×	Caching	×	✓	Simulation
[27]	Live	Hybrid-Based	Caching	Caching, TR	Caching, TR	✓	✓	Real Testbed
ALIVE	Live	Hybrid-Based	Caching	Caching, TR	Caching, TR, SR	✓	✓	Real Testbed

performance. However, some studies reveal that the ALTO protocol still has drawbacks, and its usage is limited [36].

As another well-known peer-assisted product, Akamai has a NetSession Interface [22] that supports peer-CDN cooperation. Furthermore, the Real-Time Media Flow Protocol (RTMFP) is a protocol initially developed by Adobe and is now opened up for multimedia delivery through both client-server and P2P models over the Internet [37], [38]. However, both mentioned solutions require software to be installed on the clients' devices. Our work is different from the above approaches because we do not modify the client side to have a system compatible with any ABR algorithm. In addition, we present a hybrid P2P-CDN system to use not only the storage and bandwidth resources of peers but also their computational resources (for running TR and SR functions), besides CDN servers to enhance users' QoE and network utilization.

C. Hybrid P2P-CDN NAVS Solutions

In the last decade, both academia and industry have considered the idea of combining P2P and CDN streaming to create a novel business model for OTT video delivery services. A P2P-CDN NAVS is a hybrid architecture that aims to use the advantages of both P2P and CDN systems. In such systems, the most challenging research question is: “*How should the traditional CDN and P2P networks be integrated to enable OTT providers to efficiently employ users' resources and benefit from both CDN and P2P networks' advantages?*” Hybrid P2P-CDN systems generally contain *three* main components: (i) the media servers (*i.e.*, origin and CDN servers) for distributing the video contents to the peers, (ii) peers that stream identical video contents with the same quality, and (iii) a tracker server including a matching table to find the best peers based on the considered policy (*e.g.*, minimum latency) that are watching the same video content and at the same quality level.

Nacakli and Tekalp [14] present a novel hybrid P2P-CDN service hosted at SDN-enabled edge data centers. Their proposed approach does not include any edge-based tracker server, such as our proposed VTS server between peers and CDN. Nevertheless, their solution incorporates CDN servers at the edge network and leverages CDN caching as a core function of their architecture. Ma et al. [23] propose machine

learning-based approaches for hybrid P2P-CDN systems that enable their trackers to perform peer selection. However, their system does not employ edge-supported approaches and does not use peers' computational resources. In our previous works [27], [39], we present hybrid P2P-CDN architectures that employ an online learning model to leverage the bandwidth and computational resources of CDNs, edge servers, or peers for serving live video requests. However, peers lack super-resolution capability. Borcoci et al. [40] propose a virtual content-aware network layer that enables content-awareness within the network but lacks CDN and edge integration. Some works like [24], [26] customize HAS players and equip them with a prefetching module to propose such a hybrid system in order to reduce CDN bandwidth usage and transmission costs.

Table III summarizes the differences between ALIVE and other hybrid P2P-CDN NAVS frameworks. None of the existing hybrid P2P-CDN video streaming frameworks proposes a system that (i) uses peers' potential idle computational resources for serving HAS clients through running video TR and SR; (ii) virtualizes edge trackers with caching and transcoding functionalities to be deployable on any network appliance; (iii) is compatible with all ABR policies without HAS player modification; and (iv) is implemented and evaluated in a large-scale environment.

III. SYSTEM MODEL OVERVIEW

In this section, we first discuss the problem statement (Section III-A) and then introduce the ALIVE multi-layer architecture (Section III-B). Next, we formulate the problem as an optimization model and analyze its time complexity in Section III-C.

A. Problem Statement

As discussed in Section II, most existing hybrid-based NAVS systems use peers' bandwidth and caching resources without employing edge server assistance. However, utilizing all possible peers' resources (*i.e.*, computational, caching, and bandwidth) and controlling such hybrid systems through edge servers augmented with caching and transcoding can satisfy different users' demands and requirements, improve network utilization, and decrease network costs. Imagine a hybrid P2P-CDN system, including multiple peers, some of which are

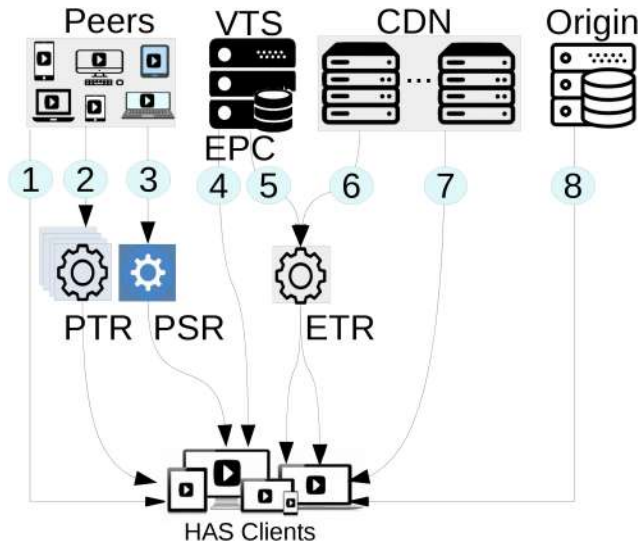


Fig. 1. ALIVE action tree (ETR := Edge Transcoding; PTR := Peer Transcoding; PSR := Peer Super-Resolution).

enabled to run peer transcoding (PTR) and/or peer super-resolution (PSR) functions. Moreover, a *Virtual Tracker Server* (VTS) is placed at the edge of the network and augmented with edge transcoding (ETR) and partial caching (EPC) functions. (We will elaborate on this in more detail in Section III-B.) Moreover, consider multiple CDN servers containing various parts of video sequences and an origin server including all video segments in various representations. In such a system, as depicted in Fig. 1, a bunch of live requests is controlled by the VTS (by running a decision-making strategy, *i.e.*, an optimization model or a heuristic algorithm), and appropriate decisions are taken to serve all clients' requests with suitable policies. The ALIVE system aims at achieving the most suitable solutions to respond to the following vital questions:

- 1) Where is the best place (*i.e.*, adjacent peers, VTS, CDN servers, or origin server) in terms of the lowest latency for fetching each client's requested content quality level from, while efficiently utilizing the available resources?
- 2) What is the best approach for responding to the requested quality level, *i.e.*, fetch, transcode it from a higher quality (using PTR or ETR), or upscale it from a lower resolution (running PSR) considering peers, edge, and CDN servers' resource limitations?

To this end, ALIVE considers various network nodes' (*i.e.*, peer, edge, CDN) resource limitations and peers' stabilities (in the sense that the least recent joining time is interpreted as the highest stability), and answers the above questions by utilizing actions from the *action tree* shown in Fig. 1 (action numbering as in the figure):

- 1) Use the P2P network and transmit the requested quality representation directly from the best adjacent peer with maximum stability (*i.e.*, the least recent joining time).
- 2) Use the PTR function to transcode the requested quality representation from a higher quality at the peer or the best adjacent peer ("best" in terms of stability, available processing, and available bandwidth) and then forward it to the peer.

- 3) Use the PSR function to upscale the requested resolution representation from a lower resolution received from the most stable adjacent peer in the P2P network.
- 4) Use the EPC function to fetch the requested representation directly from the VTS.
- 5) Use the ETR function to transcode the requested quality representation from a higher quality at the VTS and then transmit it from the VTS to the peer.
- 6) Fetch a higher quality representation from the best CDN server (*i.e.*, in terms of bandwidth), run the ETR function to transcode it at the VTS, and then transmit it from the VTS to the peer.
- 7) Fetch the requested representation directly from the best CDN server (*i.e.*, highest available bandwidth).
- 8) Fetch the requested representation from the origin server.

Note that when actions 7 and 8 are chosen, the VTS server acts as a proxy server between the clients and the CDN or origin servers. As discussed in Section II-A, the SR output is an uncompressed video representation. To prevent additional compression and transmission latencies for reconstructed representations, the proposed action tree always uses the PSR function at the last mile (on peers), where a peer plays out the uncompressed representation immediately after construction (see Section V-A for further explanation). Furthermore, since the VTS server primarily focuses on identifying and facilitating peer discovery, it only assists peers in identifying and connecting to appropriate peers with the desired content and sufficient resources when actions 1–3 are chosen. Once the connection is established between the requesting peer and the content-serving peers, the video stream is transmitted directly between those peers, bypassing the VTS server. Consequently, the involvement of VTS in these actions is limited to the initial coordination phase.

B. ALIVE Architecture

Regarding our discussion above and the problem statement, we propose the ALIVE multi-layer architecture, depicted in Fig. 2, including *four* core layers: *Media Origin*, *CDN*, *Edge*, and *P2P*.

- 1) *Media Origin Layer*. In this layer, the raw live videos are encoded and packaged into DASH format, then are stored on the origin server. Note that this layer is able to package the encoded videos to other formats like HLS or *Common Media Application Format* (CMAF).
- 2) *CDN Layer*. This layer includes a group of CDN servers (either OTT servers or a purchased service from CDN providers), each of which holds various parts of videos. Inspired by the *Consumer Technology Association* CTA-5004 standard [31], [32], CDN servers periodically inform the edge layer about their cache occupancy (*i.e.*, availability of the stored segments and their available representations) via *Common Media Server Data* (CMSD) messages.
- 3) *Edge Layer*. This layer leverages NFV and edge computing to present *Virtual Tracker Servers* (VTSs) positioned close to base stations (*e.g.*, gNodeB in 5G). Note that, in the proposed system, during a live session, clients'

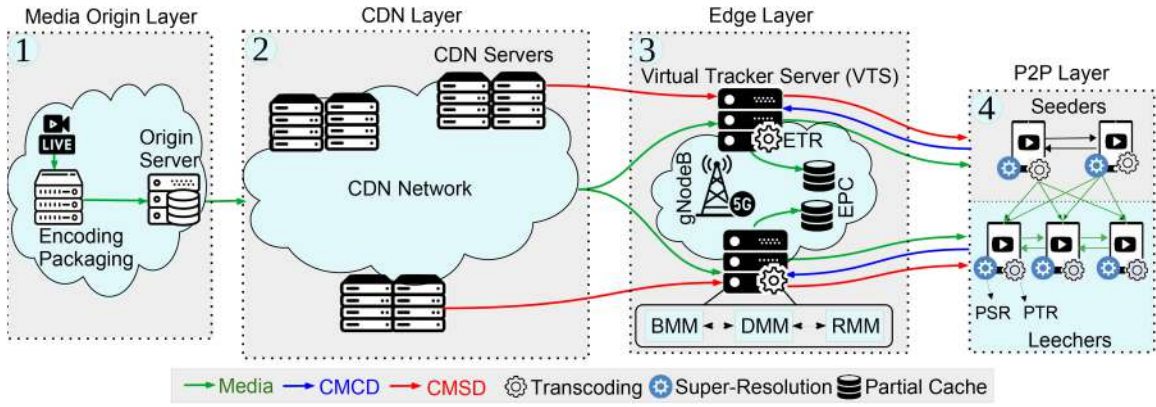


Fig. 2. ALIVE multi-layer architecture.

requests are directed to a VTS, and then they get responses based on the VTS's decisions. As shown in Fig. 2, a VTS is equipped with the following modules: (i) *Edge Transcoding Module* (ETR) to transcode segments from an existing higher quality representation to the demanded quality representation; (ii) *Edge Partial Cache Module* (EPC) that stores a limited number of segments; (iii) *Bandwidth Monitoring Module* (BMM) that monitors the available bandwidth between the VTS and other servers (*i.e.*, CDN and origin server) and then feeds the DMM module; (iv) *Resources Monitoring Module* (RMM) which is designed to monitor the system frequently, collect CMCD/CMSD messages, obtain precise information about the peers' available resources (*e.g.*, available bandwidth, computational, and power resources), find peers' joining/leaving times, and notify the DMM module; (v) *Decision-Making Module* (DMM) that operates an optimization model or a heuristic algorithm to determine appropriate solutions for the three questions listed in Section III-A. For this purpose, DMM employs the information provided by BMM and RMM and finds the best action from the *action tree* (Fig. 1). It is worth noting that the network slicing method is utilized to design an independent logical network-assisted video streaming instance over the common edge layer; therefore, the edge layer can be used to serve various network services, *e.g.*, video streaming services with various requirements.

- 4) *P2P Layer*. Given the continuous growth in smart devices' capabilities, *e.g.*, high-bandwidth access to the Internet, energy resources, and hardware-accelerated methods for running some computation-intensive tasks, ALIVE employs the peers' idle resources to provide (i) video *Peer Transcoding* (PTR) and (ii) *Peer Super-Resolution* (PSR) approaches, besides their tiny cache and idle upload bandwidth for video transmission. Like most hybrid P2P-CDN schemes, we construct the P2P layer based on the tree-mesh structure, including two types of peers: *Seeders* and *Leechers*. In this scheme, seeders' requests can be served by all nodes (*i.e.*, CDNs, origin, edge, or other seeders) except leechers, while all nodes can serve leechers' requests. Inspired by the

CTA-5004 standard [31], peers periodically inform the edge layer about their cache occupancy and available resources (*i.e.*, battery, computational, and bandwidth) through *Common Media Client Data* (CMCD) messages and receive updates from the edge layer via CMSD messages.

C. MILP Optimization Model

Let set $\mathcal{C}$ denote n CDN servers and an origin server, where $c = 0$ means the origin server (see Table II for a summary of the notation). Moreover, let us consider $\mathcal{P}$ as the set of m peers, consisting of s seeders and l leechers in two subsets $\mathcal{P}_1$ and $\mathcal{P}_2$, respectively. In addition, let set $\mathcal{V}$ define a set of VTSs. As discussed earlier, the DMM frequently receives CMCD/CMSD and bandwidth updates from the RMM and BMM modules. Considering this information, let $\alpha_{i,j}^r$ denote the availability of representation r of a segment demanded by peer j at node $i \in \{\mathcal{C} \cup \mathcal{V} \cup \mathcal{P}\}$. In addition, $\omega_{i,j}$ shows the available bandwidth between node i and peer j accordingly. When a peer j requests a representation $r^* = (x^*, y^*)$, it is possible to receive it with the exact requested quality x^* and resolution y^* from all nodes, with a lower resolution $x^*, y < y^*$ (to reconstruct it by the PSR function) or with a higher bitrate level $x > x^*, y^*$ (to rebuild it by the PTR function) from adjacent peers. In fact, if r^* is available in the cache of node i , r^* can be directly delivered to j . Otherwise, since both edge and peer nodes are able to run the TR function, the demanded representation can be reconstructed from a higher-quality representation. As another option, the requested representation can also be built by the PSR function from a lower resolution. Therefore, for each peer's request, we define $\mathcal{R}_j = \{(x^*, y_{\min}^*), \dots, (x^*, y^* - 1), (x^*, y^*), (x^* + 1, y^*), \dots, (x_{\max}^*, y^*)\}$ as the set of possible representations (pairs of (quality, resolution)), for serving requested representation $r^* = (x^*, y^*)$. Note that $y_{\min}^*$ and $x_{\max}^*$ indicate the minimum resolution and maximum quality for the requested segment, respectively.

In the following, we propose an MILP optimization model that decides an optimal action for each request issued by each peer, *e.g.*, $j \in \mathcal{P}$, based on the given information in such a way that the total latency and network cost are minimized. To this

end, the following aspects are mandated to be considered for each request to force the model to: (i) ensure that each request is served through only one action; (ii) select the best action in terms of latency (i.e., fetching, fetching/TR, or fetching/SR) and network costs (i.e., bandwidth and computation) among all possible actions; (iii) not violate the given available resource constraints by the chosen action; (iv) not violate rules defined by stakeholders. To address these aspects, we define five groups of constraints, described as follows.

(i) *Action Detector Constraint*: This constraint selects a suitable action from the proposed action tree (Fig. 1) in response to peer j 's request. Let $t \in \mathcal{T}$ define the TR status, where $t = 0$ indicates that the requested representation is transferred without transcoding, while $t = 1$, $t = 2$, or $t = 3$ denote that the requested representation is transcoded from a higher quality representation by the VTS's ETR, by the local peer's PTR, or by an adjacent peer's PTR, respectively. Moreover, $s \in \mathcal{S}$ illustrates the SR status, where $s = 1$ means the demanded representation is constructed from a lower resolution representation by the local peer's PSR, otherwise $s = 0$. Therefore, Eq. (1) determines proper values of the binary decision-making variables $D_{i,j}^{r,t,s}$ and forces the model to opt for only one action for each request issued by peers, where $D_{i,j}^{r,t,s} = 1$ indicates node i (i.e., CDN, origin, VTS, or peer) replies the representation r requested by peer j through TR status t and SR status s (see Table II for notations):

$$\sum_{i \in \{\mathcal{C} \cup \mathcal{V} \cup \mathcal{P}\}} \sum_{t \in \mathcal{T}} \sum_{s \in \mathcal{S}} \sum_{r \in \mathcal{R}_j} D_{i,j}^{r,t,s} \times \alpha_{i,j}^r = 1, \forall j \in \mathcal{P} \quad (1)$$

where $\alpha_{i,j}^r$ is used to assure the representation r requested by peer j is available in node i .

(ii) *Latency Constraints*: This group of constraints calculates the serving latency required for answering peers' requests. The first equation, i.e., Eq. (2), determines the transmitting latency $\mathcal{L}_{i,j}^r$, for transmitting representation r from each node i to peer j :

$$\sum_{t \in \mathcal{T}} \sum_{s \in \mathcal{S}} D_{i,j}^{r,t,s} \times \delta_{i,j}^r \leq \mathcal{L}_{i,j}^r \times \omega_{i,j}, \quad \forall j \in \mathcal{P}, i \in \{\mathcal{C} \cup \mathcal{V} \cup \mathcal{P}\} \setminus \{j\}, r \in \mathcal{R}_j \quad (2)$$

where $\delta_{i,j}^r$ indicates the size (in bytes) of representation r requested by the peer $j \in \mathcal{P}$. In addition, Eq. (3) measures the required computational latency $\tau_{i,j}^r$, i.e., TR time at node $i \in \{\mathcal{V} \cup \mathcal{P}\}$ or SR time at node $i = j$, in case of serving the representation $r \in \mathcal{R}_j$ requested by $j \in \mathcal{P}$, from a higher quality or a lower resolution representation, respectively.

$$\sum_{t \in \mathcal{T}} \sum_{s \in \mathcal{S}} \sum_{r \in \mathcal{R}_j} D_{i,j}^{r,t,s} \times \mu_{i,j}^r \leq \tau_{i,j}^r, \forall j \in \mathcal{P}, i \in \{\mathcal{P} \cup \mathcal{V}\} \quad (3)$$

Hence, the total serving latency, i.e., Ψ , is the sum of transmitting latency ($\mathcal{L}_{i,j}^r$ in Eq. (2)) plus computational latency ($\tau_{i,j}^r$ in Eq. (3)) and can be expressed as follows:

$$\sum_{i \in \{\mathcal{C} \cup \mathcal{V} \cup \mathcal{P}\}} \sum_{j \in \mathcal{P}} \sum_{r \in \mathcal{R}_j} \mathcal{L}_{i,j}^r + \tau_{i,j}^r \leq \Psi \quad (4)$$

(iii) *Policy Constraints*: These constraints apply essential policies to the model to prevent choosing incorrect actions. Eq. (5) forces the model to fetch the exact representation r^*

TABLE II
NOTATION FOR ALIVE

Notation	Description
Input Parameters	
$\mathcal{C}$	Set of n CDN servers and an origin server (i.e., $c = 0$)
$\mathcal{P}$	Set of m peers including s seeders and l leechers in subsets $\mathcal{P}_1$ and $\mathcal{P}_2$, respectively
$\mathcal{V}$	Set of Virtual Tracer Servers (VTSs)
$\mathcal{R}_j$	Set of possible representations (i.e., (quality, resolution)) for serving representation $r^* = (x^*, y^*)$ requested by $j \in \mathcal{P}$, where $\mathcal{R}_j = \{(x^*, y_{min}^*), \dots, (x^*, y^*), \dots, (x_{max}^*, y^*)\}$ and y_{min}^*, x_{max}^* are the available minimum resolution and maximum quality for the demanded segment, respectively
$\mathcal{T}$	Set of possible transcoding statuses, where $\mathcal{T} = \{0, 1, 2, 3\}$; $t = 1$, $t = 2$, or $t = 3$ if the requested representation is transcoded from a higher quality representation $r \in \mathcal{R}_j$ by the ETR, by the peer's PTR, or by an adjacent peer's PTR, respectively; otherwise $t = 0$
$\mathcal{S}$	Set of possible SR statuses; $\mathcal{S} = \{0, 1\}$; $s = 1$ if a peer runs the PSR function to build the requested representation; otherwise $s = 0$
$\mathcal{X}$	Set of ρ peer regions
$\mathcal{Q}$	Set of queues of clients' requests (i.e., representations) in the VTS; $\mathcal{Q} = \{\mathcal{Q}_{r_{min}}, \dots, \mathcal{Q}_{r_{max}}\}$
$\alpha_{i,j}^r$	Available representations in $i \in \{\mathcal{C} \cup \mathcal{V} \cup \mathcal{P}\}$; $\alpha_{i,j}^r = 1$ means node i hosts representation r requested by peer $j \in \mathcal{P}$; otherwise $\alpha_{i,j}^r = 0$
$\omega_{i,j}$	Available bandwidth on path between $i \in \{\mathcal{P} \cup \mathcal{V} \cup \mathcal{C}\} \setminus \{j\}$, and $j \in \mathcal{P}$
$\theta_{i,j}^r$	Required computational resources (i.e., CPU time in seconds) for building $r \in \mathcal{R}_j$ requested by $j \in \mathcal{P}$ through TR in $i \in \{\mathcal{V} \cup \mathcal{P}\}$ or via PSR in $i = j$, respectively
$\eta_{i,j}^r$	Required power (in milliamperes-hour) for constructing $r \in \mathcal{R}_j$ requested by $j \in \mathcal{P}$ through TR in $i \in \mathcal{P}$ or SR in $i = j$
$\mu_{i,j}^r$	Required time (in seconds) for building $r \in \mathcal{R}_j$ requested by $j \in \mathcal{P}$ through TR in $i \in \{\mathcal{V} \cup \mathcal{P}\}$ or via SR in $i = j$
$\delta_{i,j}^r$	Size of segment in representation r (in bytes) requested by $j \in \mathcal{P}$
λ_j	Bitrate for representation $r \in \mathcal{R}_j$ requested by $j \in \mathcal{P}$
Ω_i, ϕ_i	Available computation resources (available CPU) of $i \in \{\mathcal{V} \cup \mathcal{P}\}$ and available power resources of $i \in \mathcal{P}$, respectively
Δ_{co}, Δ_{bw}	Computational cost per CPU core per second and bandwidth cost per bit per second, respectively
β	Adjustable weight coefficient for the latency, where $1 - \beta$ is the network costs coefficient
Variables	
$D_{i,j}^{r,t,s}$	Binary variable where $D_{i,j}^{r,t,s} = 1$ shows node $i \in \{\mathcal{C} \cup \mathcal{V} \cup \mathcal{P}\}$ transmits representation $r \in \mathcal{R}_j$ requested by peer $j \in \mathcal{P}$ with TR status t and SR status s , otherwise $D_{i,j}^{r,t,s} = 0$
$\tau_{i,j}^r$	Required computation time to build the representation $r \in \mathcal{R}_j$ requested by $j \in \mathcal{P}$ via TR at node $i \in \{\mathcal{V} \cup \mathcal{P}\}$ or via PSR at $i = j$
$\mathcal{L}_{i,j}^r$	Required transmitting time of representation $r \in \mathcal{R}_j$ in response to peer $j \in \mathcal{P}$ from node $i \in \{\mathcal{C} \cup \mathcal{V} \cup \mathcal{P}\} \setminus \{j\}$
Ψ	Total serving latency, including $\tau_{i,j}^r$ and $\mathcal{L}_{i,j}^r$
Π, Λ	Network computational and bandwidth costs, respectively
ξ	Total network costs consisting of Π and Λ

from the origin or CDN servers when one of them is chosen to serve peer j .

$$\sum_{i \in \mathcal{C}} \sum_{r \in \mathcal{R}_j} D_{i,j}^{r,t=0,s=0} \times r = r^* \quad \forall j \in \mathcal{P} \quad (5)$$

Note that $i = 0$ in Eq. (5) indicates that the origin server is selected to serve the requested representation r . Moreover, since the SR module can only be run on the local peer to serve the requested representation r , Eq. (6) prevents other nodes ($i \neq j$) from using their computational resources for running SR:

$$\sum_{i \in \{\mathcal{C} \cup \mathcal{V} \cup \mathcal{P}\} \setminus \{j\}} \sum_{r \in \mathcal{R}_j} \sum_{t \in \mathcal{T}} D_{i,j}^{r,t,s=1} \times r = 0, \quad \forall j \in \mathcal{P} \quad (6)$$

Furthermore, Eq. (7) controls seeders not to fetch the requested representation r from a leecher:

$$\sum_{i \in \mathcal{P}_2} \sum_{t \in \mathcal{T} \setminus \{1\}} \sum_{s \in \mathcal{S}} \sum_{r \in \mathcal{R}_j} D_{i,j}^{r,t,s} \times r = 0, \quad \forall j \in \mathcal{P}_1 \quad (7)$$

The final constraint of this group prevents peers from serving the requested representation r by employing both PTR and PSR simultaneously:

$$\sum_{r \in \mathcal{R}_j} D_{i,j}^{r,t=2,s=1} \times r = 0, \quad \forall i, j \in \mathcal{P}, \text{ where } i = j \quad (8)$$

(iv) *Resource Constraints*: This group of constraints guarantees that the required resources for constructing and/or

transmitting the requested segments will not violate the available resources. Therefore, Eq. (9) ensures that the required bandwidth for transmitting representation r on the link between nodes i and j will respect the available bandwidth:

$$\sum_{t \in \mathcal{T}} \sum_{s \in \mathcal{S}} \sum_{r \in \mathcal{R}_j} D_{i,j}^{r,t,s} \times \lambda_j^r \leq \omega_{i,j}, \quad \forall j \in \mathcal{P}, i \in \{\mathcal{C} \cup \mathcal{V} \cup \mathcal{P}\} \setminus \{j\} \quad (9)$$

where λ_j^r is the bitrate associated with representation r . Furthermore, Eq. (10) restricts the maximum processing capacity required for the ETR function to the available computational resource on the VTS server i (denoted by Ω_i):

$$\sum_{j \in \mathcal{P}} \sum_{r \in \mathcal{R}_j} D_{i,j}^{r,t=1,s=0} \times \theta_{i,j}^r \leq \Omega_i, \quad \forall i \in \mathcal{V} \quad (10)$$

where $\theta_{i,j}^r$ is the required computational resources (*i.e.*, CPU time in seconds) for running the ETR function to achieve demanded representation r . Similarly, Eq. (11) limits the maximum processing capacity of peer i for serving adjacent peers' requests through PTR and serving its own requests through the PTR or PSR functions to the available computational resource (*i.e.*, Ω_i):

$$\begin{aligned} & \underbrace{\sum_{r \in \mathcal{R}_i} D_{i,i}^{r,t=2,s=0} \times \theta_{i,i}^r}_{\text{PTR computational usage for own requests}} \\ & + \underbrace{\sum_{r \in \mathcal{R}_i} D_{i,i}^{r,t=0,s=1} \times \theta_{i,i}^r}_{\text{PSR computational usage for own requests}} \\ & + \underbrace{\sum_{j \in \mathcal{P} \setminus \{i\}} \sum_{r \in \mathcal{R}_j} D_{i,j}^{r,t=3,s=0} \times \theta_{i,j}^r}_{\text{PTR computational usage for adjacent peers}} \leq \Omega_i \quad \forall i \in \mathcal{P} \end{aligned} \quad (11)$$

In addition, Eq. (12) restricts the maximum power resources of peer i required for running the PTR or PSR functions to the available power resources (shown by ϕ_i):

$$\begin{aligned} & \underbrace{\sum_{r \in \mathcal{R}_i} D_{i,i}^{r,t=2,s=0} \times \eta_{i,i}^r}_{\text{PTR power usage for own requests}} + \underbrace{\sum_{r \in \mathcal{R}_i} D_{i,i}^{r,t=0,s=1} \times \eta_{i,i}^r}_{\text{PSR power usage for own requests}} \\ & + \underbrace{\sum_{j \in \mathcal{P} \setminus \{i\}} \sum_{r \in \mathcal{R}_j} D_{i,j}^{r,t=3,s=0} \times \eta_{i,j}^r}_{\text{PTR power usage for adjacent peers}} \leq \phi_i \quad \forall i \in \mathcal{P} \end{aligned} \quad (12)$$

where $\eta_{i,j}^r$ is the required power (in milliampere-hour) for building the representation r requested by peer j on peer i through PTR or PSR functions.

(v) *Cost Constraints*: The final group of constraints formulates the total network costs at the infrastructure level, including bandwidth and computational costs. For this purpose, Eq. (13) calculates the network bandwidth cost (*i.e.*, Λ in dollars) for transmitting segments on the link between server $i \in \{\mathcal{C} \cup \mathcal{V}\}$ and peer $j \in \mathcal{P}$ as follows:

$$\Delta_{bw} \times \sum_{t \in \mathcal{T}} \sum_{s \in \mathcal{S}} \sum_{r \in \mathcal{R}_j} D_{i,j}^{r,t,s} \times \lambda_j^r \leq \Lambda, \quad \forall j \in \mathcal{P}, i \in \{\mathcal{C} \cup \mathcal{V}\} \quad (13)$$

where Δ_{bw} refers to the bandwidth cost per bit per second. Similarly, Eq. (14) measures the total required computational cost Π required for the ETR function at the VTS $v \in \mathcal{V}$:

$$\Delta_{co} \times \sum_{j \in \mathcal{P}} \sum_{r \in \mathcal{R}_j} D_{i,j}^{r,t=1,s=0} \times \theta_{i,j}^r \leq \Pi, \quad \forall i \in \mathcal{V} \quad (14)$$

where Δ_{co} indicates computational resource cost per CPU core per second. Since bandwidth and computational costs are essential parts of nowadays' networks, we thus represent the total network costs (*i.e.*, ξ) as follows:

$$\Pi + \Lambda \leq \xi \quad (15)$$

Note that in the case of selecting actions 1–3 (*i.e.*, serving requests via a P2P network), these constraints are not taken into account by P2P-CDN architectures such as ALIVE. This is because users who voluntarily join and participate in a P2P-CDN network are aware that their resources may be used for content delivery. Therefore, they implicitly agree to share their resources without expecting direct compensation for computational or bandwidth costs. On the other hand, given the prevalence of smartphones among peers today, many peers have constrained resources and power capabilities, which might deter them from participating in running video transcoding or sharing media with other peers. P2P-CDN providers often incentivize users to participate in the network by offering rewards or benefits based on the “tit-for-tat” strategy, *e.g.*, faster access to content or reduced costs to consume content. However, it should be noted that ALIVE is compatible with any pre-established P2P network, and the development of incentive mechanisms is beyond the scope of ALIVE.

Furthermore, when transcoding video sequences in adjacent peers, it is essential to employ appropriate privacy policies. This is because video transcoding can provide that peer with exact insights into the content requested by other peers, which is against the user's privacy. However, introducing security or privacy methods is beyond the scope of the design of ALIVE.

Optimization Problem: As described in Section I, a HAS client always operates an ABR algorithm to estimate the network's bandwidth. Indeed, it measures the time between requesting a segment's representation to download and receiving its last packet. Thus, minimizing the latency in the optimization model affects the HAS clients' performance and improves users' QoE directly. On the other hand, the optimization model must be able to be adjusted by the network operator based on their business models. Therefore, making a trade-off between latency and network costs in the objective function makes the model adaptable to the operators' desired policies. For this purpose, the MILP model (16) considers multiple objectives to minimize the clients' total latency as well as the network costs:

$$\begin{aligned} & \text{Minimize } \beta \times \frac{\Psi}{\Psi^*} + (1 - \beta) \times \frac{\xi}{\xi^*} \\ & \text{s.t. constraints Eq. (1) – Eq. (15)} \\ & \text{vars. } \mathcal{T}_{i,j}^r, \tau_{i,j}^r, \Psi, \xi, \Pi, \Lambda \geq 0, D_{i,j}^{r,t,s} \in \{0, 1\} \end{aligned} \quad (16)$$

By minimizing the objective function (16), an optimal action will be selected for each request issued by peer $j \in \mathcal{P}$. The

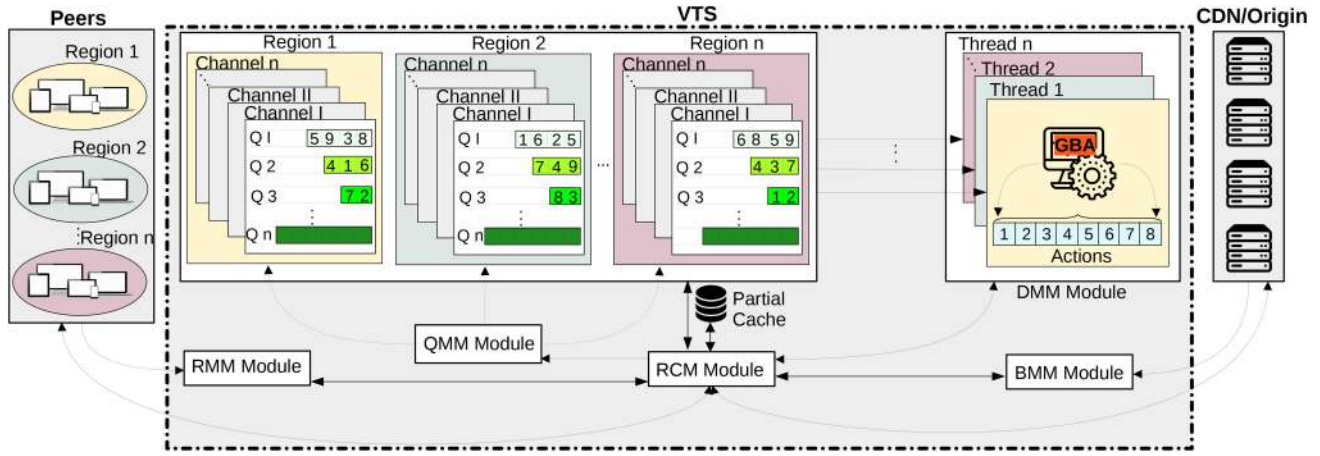


Fig. 3. Proposed ALIVE heuristic workflow.

weighting coefficient (β) enables operators to adjust desirable preferences for latency and network costs based on their specific policies. Since Ψ and ξ have different dimensions (i.e., seconds and currency unit, respectively), a normalization method is required to form the objective function. Hence, we normalize each factor by dividing it by its associated maximum value (i.e., Ψ^* and ξ^* , respectively) calculated over all possible actions. To measure Ψ^* , we run the MILP model with $\beta = 0$ to achieve the minimum network costs, in other words, the maximum serving latency. Similarly, we set $\beta = 1$ to measure ξ^* . The calculated Ψ^* and ξ^* values are then reused to run the model for different sets of β .

Theorem 1: The proposed MILP model (16) is an NP-hard problem.

Proof: Utilizing the introduced binary and non-binary variables drives the optimization model (16) to be an MILP model. Employing methods introduced by [41], the proposed MILP model can be reduced to the *Multiple Knapsack Problem* (MKP), which is a well-known NP-hard problem [42], in polynomial time. ■

IV. HEURISTIC ALGORITHM

The introduced optimization model (16) suffers from high time complexity, which makes it impractical for large-scale scenarios. To cope with this issue, we propose a lightweight *Greedy-Based Algorithm* (GBA), which is run by the DMM module instead of the MILP model (16). Before explaining the details of GBA, let us describe the new modules, workflow, and time slot structure used by each VTS server to form the proposed heuristic strategy in the following subsection.

A. Heuristic Workflow

As shown in the heuristic workflow (Fig. 3), in addition to the modules introduced in Section III-B, each VTS server hosts multiple *queues of clients' requests* (i.e., representations), one per peer region, live video channel, and each channel's representation (bitrate, resolution). Moreover, each VTS is augmented with a *Request Controller Module* (RCM) and a *Queueing Manager Module* (QMM). A time slot structure,

Algorithm 1: RCM Module

Input: *requests*, *RMM_info*, *BMM_info*, *on_the_fly*

```

1 Ext_info ← ExtractInfo(requests)
2 Agg_reqs ← Agg(requests, Ext_info)
3 queues ← QMM(Agg_reqs)
4 for each reg in Regions do
5   Sorted_queuesreg ← Sort(queuesreg, Ext_inforeg)
6   One DMM() thread takes care of each region reg
7   for each req in Sorted_queuesreg do
8     if req ∈ on_the_fly then
9       HoldReq(req)
10    else
11      on_the_fly.add(req)
12      opt_node, opt_action ← DMM(req, RMM_info,
13                                BMM_info,  $\beta$ )
14      ServeRequest(req, opt_node, opt_action)
15      Update()

```

including two intervals, (i) *Collecting Interval* (CI) and (ii) *Serving Interval* (SI), is also utilized by each VTS.

In the CI interval, the RMM and BMM modules deliver obtained CMCD and CMSD messages, measured available resources (i.e., bandwidth, power, computation, joining/leaving times), and queues' information to the RCM module. The pseudo-code of the RCM module is shown in Alg. 1, where it receives HTTP clients' requests, the RMM and BMM modules' information, and an *on_the_fly* list (holding requests currently being served) as input parameters. It calls the *ExtractInfo* function to extract regions (based on IP addresses), demanded channels, and representations from the incoming HTTP requests and saves the extracted information in the *Ext_info* list (line 1). It then aggregates the incoming HTTP requests and the extracted information (i.e., region/channel/bitrate) via the *Agg* function and forwards the outputs stored in *Agg_reqs* to the QMM module (lines 2–3).

Since more than one queue can proceed and might violate all/several resource constraints (e.g., the bandwidth, computation, or power limitations), they must be assessed in priority order. For the sake of simplicity, in this work, the queue with a higher number of requests is prioritized for service. However, the proposed heuristic solution can be easily extended to

Algorithm 2: GBA Algorithm

Input: $req, RMM_info, BMM_info, \beta$
Output: opt_node, opt_action

```

1  $opt\_node \leftarrow \{\emptyset\}, opt\_action \leftarrow \{\emptyset\}, opt\_obj \leftarrow 1000000$ 
2  $Fsb\_nodes \leftarrow FsbNodes(req, RMM\_info, BMM\_info)$ 
3 for each  $j \in Fsb\_nodes$  do
4    $Fsb\_actions \leftarrow FsbActions(j, req)$ 
5    $act, obj \leftarrow CostFunction(j, req, Fsb\_actions,$ 
6      $RMM\_info, BMM\_info, \beta)$ 
7   if  $opt\_obj > obj$  then
8      $opt\_node \leftarrow j, opt\_action \leftarrow act, opt\_obj \leftarrow obj$ 
9 return  $opt\_node, opt\_action$ 

```

accommodate other conditions or to employ other queuing methods, such as prioritizing the requests coming from peers that may be critically low on buffered content. Hence, the *Sort* function is used to store the region's queues in the *SortedQueue* list (line 5). The algorithm then starts to serve requests req in the highest priority queue by comparing it to the requests in the *on_the_fly* list. The *HoldReq* function is called when req is in the *on_the_fly* list (i.e., req has been issued by another peer and still is in progress) to hold the request to prevent sending identical requests if a response to the request is in flight from the CDN/origin server (lines 7–9).

When req is not found in the *on_the_fly* list, that means req is a completely new one, and the *on_the_fly* list must be updated by req to be processed (line 11). Considering the req , the received BMM_info and RMM_info structures, and the coefficient value β , the RCM calls the *DMM* module to determine the optimal node and action based on the objective function in Eq. (16) and the adjusted weight coefficient (i.e., β) for the request req . The results are stored in opt_node and opt_action (line 12). Moreover, the *ServeRequest* function is called to communicate with the peers, a CDN or the origin server concerning the decisions made by the DMM, and to serve the peer's request req (line 13). Finally, the RCM updates the *available resources*, stores popular segments fetched from a CDN or the origin server into the *partial cache*, and updates the *on_the_fly* list via the *Update* function (line 14).

Considering the information provided in the CI interval (i.e., available information on resources and queues of clients' requests (i.e., representations) provided by the RCM and QMM modules), the DMM in the SI interval must run multiple threads of the GBA algorithm (one thread per peer region) to answer the questions mentioned in Section III-A. This time slot structure will be repeated until the live streaming session ends and all queues are served. The details of the proposed heuristic algorithm are described in the following subsection.

B. Greedy-Based Algorithm (GBA)

Alg. 2 describes the pseudo-code of our proposed *greedy-based algorithm* (GBA) invoked by the *DMM* module to solve the optimization problem (16). It receives a request req , RMM_info , BMM_info , and coefficient β as inputs and returns the determined opt_node and opt_action for serving the req as outputs.

The optimal node opt_node , optimal action opt_action , and associated objective value opt_obj are initialized (lines 1).

Algorithm 3: CostFunction Function

Input: $j, req, Fsb_actions, RMM_info, BMM_info, \beta$
Output: act, obj

```

1  $act \leftarrow \{\emptyset\}, obj \leftarrow 1000000, obj\_aux \leftarrow 0$ 
2  $max\_delay, max\_cost \leftarrow$ 
3    $CalcMax(j, req, RMM\_info, BMM\_info)$ 
4 if  $j \in \mathcal{P}$  then
5   for each  $a \in Fsb\_actions$  do
6      $delay \leftarrow CalcDly(j, a, RMM\_info, BMM\_info)$ 
7      $obj\_aux \leftarrow \frac{\beta \times delay}{max\_delay}$ 
8     if  $obj > obj\_aux$  then
9        $act \leftarrow a, obj \leftarrow obj\_aux$ 
10 else
11   for each  $a \in Fsb\_actions$  do
12      $delay \leftarrow CalcDly(j, a, RMM\_info, BMM\_info)$ 
13      $cost \leftarrow CalcCst(i, j, a, RMM\_info, BMM\_info)$ 
14      $obj\_aux \leftarrow \frac{\beta \times delay}{max\_delay} + \frac{(1-\beta) \times cost}{max\_cost}$ 
15     if  $obj > obj\_aux$  then
16        $act \leftarrow a, obj \leftarrow obj\_aux$ 
17 return  $act, obj$ 

```

In the given algorithm, two auxiliary variables, namely act and obj , are utilized to aid in the determination of the optimal action and optimal objective, respectively. Moreover, the *FsbNodes* function is called to determine all feasible nodes (i.e., peers, VTS, CDNs, or origin) that hold the requested representation or convertible representations (i.e., higher or lower ones for running TR and SR, respectively), and to prepare a list of all feasible nodes in Fsb_nodes (line 2).

In the next step, first, all feasible actions for each feasible node are determined and stored in the $Fsb_actions$ list by invoking the *FsbActions* function (line 4). The algorithm then employs the *CostFunction* function (Alg. 3) to find the action with the minimum objective value among all determined feasible actions of a specific node.

The outputs of the *CostFunction* are stored in the auxiliary variables obj and act (line 5). Next, it checks whether a new minimum objective value has been determined (lines 7-8). Finally, at the end of the for loop, the best node and action with a minimum value of obj is selected and returned as opt_node and opt_action (line 9).

The *CostFunction* function (Alg. 3) calculates the objective function (16) for each node and its associated actions stored in $Fsb_actions$, respectively. In the first step, it initializes the primary variables, i.e., objective obj , action act , and auxiliary variable obj_aux (line 1). It then calls *CalcMax* function to calculate the maximum values of the delay and cost based on the strategy discussed in Section III-C and stores them in max_delay and max_cost , respectively (line 2). Afterward, it checks the node j to find if it is a neighboring peer or a node of another type (VTS, CDN, or origin). Suppose that it will be a peer; thus, Alg. 3 in a for loop measures the minimum objective value by calling the *CalcDly* function in each iteration. Finally, the determined objective and its associated action are stored in act and obj , respectively (lines 3-8). Otherwise, the same strategy is utilized to calculate the minimum objective value via calling the *CalcDly* and *CalcCst* functions (lines 9-15).

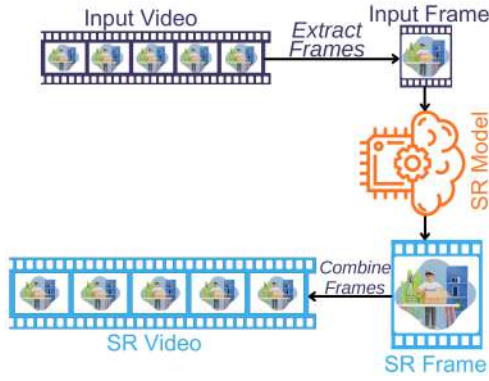


Fig. 4. Illustration of a video super-resolution pipeline.

Assume ρ , κ , and γ indicate the number of peer regions, number of live channels, and number of representations per channel. In the worst case, the time complexity of the proposed heuristic method would be $O(\rho \times \kappa \times \gamma)$ in each time slot.

V. PERFORMANCE EVALUATION

This section describes the employed SR models, explains the evaluation setup, metrics, and methods, and assesses the performance of ALIVE in four scenarios.

A. Super-Resolution Models

Super-resolution (SR) is a computational task for restoring high-frequency signals lost in the low-resolution image to obtain the high-resolution counterpart. The goal of such a process is minimizing visual degradation as much as possible while obtaining a high-resolution image [43]. Although traditional SR methods like *bilinear* or *bicubic* interpolate pixel values directly, they always suffer from visual artifacts. In recent years, attention has shifted towards machine learning-based image SR [43]. These models (e.g., *SRCNN* [44]) can learn the mapping from low-resolution image patches to high-resolution patches directly and outperform traditional SR approaches. Some learning-based SR methods focus on better visual quality, while others concentrate on making models practical by alleviating their complexity.

Inspired by the image SR, applying SR techniques for video sequences has also been popular recently. As illustrated in Fig. 4, the video frames must be extracted for the video SR process. Then the SR model can be applied to each frame or a small group of frames individually. Finally, the frames upsampled by SR must be combined to produce the reconstructed video. We run the most recent and popular video SR models, namely, *ESPCN* [45], *LiDeR* [46], *FSRCNN* [47], *EDSR* [48], *CARN* and *CARN-M* [49], *NINASR* [50], and *RCAN* [51] to choose our PSR models with maximum quality (measured by *Video Multi-method Assessment Fusion* (VMAF) [52]) and minimum imposed delay (measured by frames per second (FPS)). The VMAF-FPS graph for the PSR models is depicted in Fig. 5. As shown in Fig. 5, CARN-M provides a high VMAF score while still managing to run in real-time using customer-grade hardware. Moreover, LiDeR is

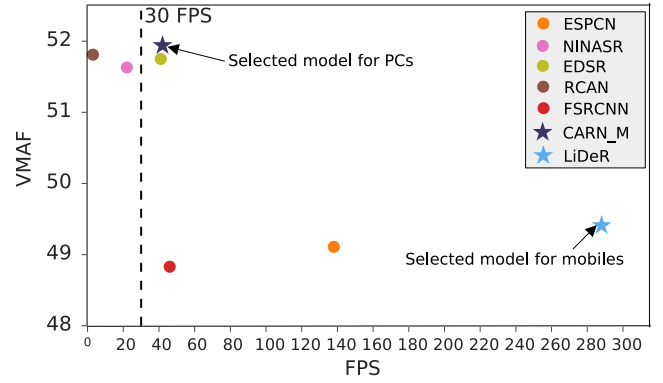


Fig. 5. Performance of different video SR models in terms of VMAF and FPS. Results are obtained with input resolution 640×360 and target resolution 1280×720 . The 30 FPS line is shown to indicate real-time execution.

a model that can run in real-time with a higher VMAF score compared to FSRCNN and ESPCN. Therefore, we use the following video PSR models in all scenarios:

- 1) *CARN for PC-type peers*: CARN [49] is proposed to use cascading residual connections in the neural network. CARN's neural network is designed based on the ResNet [53] structure. Cascading connections are added on top of the residual connections to enable information propagation between layers. These changes help CARN to process information using fewer layers. CARN achieves competitive SR performance with state-of-the-art networks while running much faster. Moreover, a lightweight version of CARN, called *CARN-M*, using a more efficient residual block structure, is designed to be applied in practical scenarios. We use it in our system on PC-type peers.
- 2) *LiDeR for mobile peers*: Due to the improvements in smartphone hardware, particularly supporting GPUs, complex deep neural networks (DNNs) models can be run on mobile devices. However, mobile battery consumption is still a constraint compared to standard desktop PCs. LiDeR [46] is designed considering the constraints of mobile devices based on a lightweight dense residual neural network to operate on decoded videos in real-time without hardware-specific optimization.

B. Testbed Design and Setup

We use a real backbone network topology called *InternetMCI* [54] to assess the effectiveness of the ALIVE framework in a realistic large-scale setting. We instantiate a cloud-based large-scale testbed on the CloudLab [55] environment. As illustrated in Fig. 6, our testbed utilizes 375 elements, each of which runs Ubuntu 20.04 LTS inside Xen virtual machines. These components are: (i) Seven groups of 50 open-source *AStream* [56] DASH players (in total 350 peers) working in a headless mode. In all experiments, it is assumed that each group corresponds to a specific region of live viewers. Each group includes five seeders and 45 leechers in a tree-mesh structure. *SQUAD* [57] and *BOLA* [58]

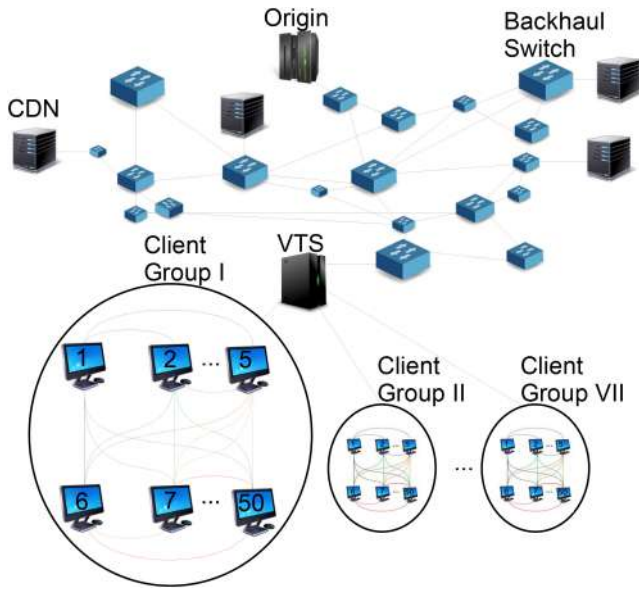


Fig. 6. ALIVE evaluation testbed.

are used in DASH clients as hybrid and buffer-based ABR algorithms, respectively. (ii) Four CDN servers, containing a total cache size of 40% of the video dataset, and an origin server, hosting all video sequences. The server machines run an Apache HTTP server and MongoDB. (iii) 19 OpenFlow (OF) backbone switches, forming the backbone topology with 45 backbone layer-2 links. (iv) A VTS server with a partial cache size of only 5% of the video sequences. Note that ALIVE is independent of the caching policy and is compatible with various caching strategies.

For the sake of simplicity, we set the Least Recently Used (LRU) scheme in all CDN and partial caches as the cache replacement policy. Furthermore, since peers can store a limited amount of content, we assume each peer can cache five segments of the videos at most. Python-based HTTP servers and the Python PuLP library with the CPLEX solver library are employed to implement the VTS and the proposed MILP model, respectively. Moreover, all modules of the VTS server, including the heuristic algorithm, are implemented in Python. Due to the limited storage in our servers, we only use five live channels (*i.e.*, CH I–CH V). Each live channel plays a unique video [59] with 300 seconds duration and two-second segments in bitrate ladder (representation set) $\{(0.089, 320p), (0.262, 480p), (0.791, 720p), (2.4, 1080p), (4.2, 1080p)\}$ [Mbps, content resolution]. However, ALIVE is not limited to this live dataset and is compatible with other live HAS contents with another segment duration and bitrate ladder. For simplicity, all scenarios assume that the channel probability is known in advance and sorted in descending order. The channel access probability is generated by a Zipf distribution [60] with the skew parameter $\alpha = 0.7$, *i.e.*, the probability of an incoming request for the i^{th} channel in each peer group is given as $prob(i) = \frac{1/i^\alpha}{\sum_{j=1}^K 1/j^\alpha}$, where $K = 5$.

We use Docker image *jrottenberg/ffmpeg* [61] to measure the segment transcoding time on the VTS. Moreover, we

execute the transcoding function via *FFmpegKit* [62] on an iPhone 11 (Apple A13 Bionic, iOS 15.3), a Xiaomi Mi11 (Snapdragon 888, Android 11), and a PC (Apple M1, MacOS 12.0.1) to measure the transcoding time on the heterogeneous P2P network. Power consumption is measured by device tools, *e.g.*, *Android Energy Profiler* and *Android Battery Manager*. Moreover, we utilize the *CodeCarbon* [63] project to measure power consumption for the PC-type clients. In the literature, the bandwidth value from an edge server to CDN servers is assumed higher than to the origin server [64], [65]. Therefore, the Linux *Wondershaper* tool [66] is employed to set 50 and 100 Mbps as the bottleneck bandwidth in different paths from the CDN and origin servers to the VTS, respectively.

To emulate the mobile network conditions and consider the dynamics of peers, we simply assume 250 peers start the experiments initially, and then every three seconds a new peer joins the session. We use a real 4G network trace [67] collected on bus rides for links between peers to edge servers in all experiments to create dynamic bandwidth for peers. The average bandwidth of this trace is approximately 3780 kbps with a standard deviation of 3190 kbps. The VTS directs the first peer to the best CDN server (in terms of lowest latency), while other participating peers can be connected on both CDN and P2P links. It should be noted that, to allow for meaningful evaluation experiments, the bandwidth of all links is known. In the real world, assessing the capacity between peers in a large-scale P2P network presents a significant challenge in itself. In hybrid P2P-CDN systems like ALIVE, most costs are related to backhaul bandwidth and edge computational costs. Therefore, other service costs are negligible compared to the aforementioned costs. The computational and bandwidth costs (*i.e.*, Δ_{co} and Δ_{bw}) are set to 0.029\$ per CPU per hour and 0.12\$ per GB, respectively [68]. The weighting parameter β and the monitoring interval are set to 0.5 and one second in all experiments, respectively. By selecting a small monitoring interval, we decrease the risk of making decisions based on outdated information and allow the system to operate with more current and relevant data. As explained in Section III-A, the most stable peers are always considered if actions 1–3 are chosen. However, in case of connection loss during the P2P exchange, the clients enter a buffering state and await the subsequent round.

C. Methods for Comparison, Scenarios, and Metrics

Methods for Comparison: In the ALIVE performance evaluation, we first investigate the SR models' behavior. Next, we analyze the performance of the proposed optimization model and heuristic algorithms. Finally, we refer to our proposed heuristic methods and present practical testbed results to evaluate ALIVE's effectiveness compared to the following baseline schemes.

- 1) *Non Hybrid (NOH)*: The NOH system is regular CDN-based streaming with no P2P support.
- 2) *Simple Edge-enabled Hybrid (SEH)*: The SEH system employs a simple VTS server without caching and transcoding capabilities. In this system, peers only can be served via one of the actions 1, 7, or 8 (Fig. 1).

TABLE III
ACRONYMS USED IN ALIVE'S EVALUATION

Acronym	Full term
GBA	Greedy-Based Algorithm
NOH	Non Hybrid Scheme
SEH	Simple Edge-enabled Hybrid Scheme
NTH	Non Transcoding-enabled Hybrid Scheme
ECT	Edge Caching/Transcoding Hybrid Scheme
NSH	Non SR-enabled Hybrid Scheme
VMAF	Video Multi-method Assessment Fusion
ETV	Execution Time Values
NOV	Normalized Objective Values
ASB	Average Segment Bitrate
AQS	Average Quality Switches
ASD	Average Stall Duration
ANS	Average Number of Stalls
ASL	Average Serving Latency
APQ	Average Perceived QoE
CHR	Cache Hit Ratio
ETR	Edge Transcoding Ratio
PTSR	Peer SR and TR Ratio
BTL	Backhaul Traffic Load
EEC	Edge Energy Consumption
NCV	Network Cost Values

- 3) *Non Transcoding-enabled Hybrid (NTH)*: Like in most works, there is no transcoding capability in this approach. In an NTH-based system, peers only can be served through one of the actions 1, 4, 7, or 8 (Fig. 1).
- 4) *Edge Caching/Transcoding Hybrid (ECT)*: ECT does not include transcoding at the peer side, and requests can be served via all actions except actions 2 and 3.
- 5) *Non SR-enabled Hybrid (NSH)*: In the NSH-based system, there is no SR feature on the peer side and requests can be served via all actions except action 2.

Note that we use our testbed with similar settings and topology in all systems for fair and robust comparisons. Moreover, all aforementioned baseline schemes execute a simplified version of the proposed GBA algorithm to answer the questions mentioned in Section III-A.

Description of Scenarios: We compare the performance of the ALIVE methods with the aforementioned baseline approaches in four scenarios.

- 1) *Scenario I*: This scenario studies the impact of changing SR models on the *perceptual quality* metrics. Moreover, it investigates the *latency*, *quality distortion*, and *energy overheads* of running computational-intensive tasks, *i.e.*, transcoding and super-resolution, on peers.
- 2) *Scenario II*: In this scenario, we provide a performance comparison of the proposed optimization model and the heuristic algorithm by conducting several experiments. These experiments study the impact of changing key parameters like the *bitrate ladder size*, the number of *peers' requests*, and *live channels* on the *optimization model and heuristic algorithms performance* metrics.
- 3) *Scenario III*: In this scenario, we run a series of experiments on the designed testbed to assess the performance of the explained systems in terms of common *QoE-related* parameters compared to other approaches.
- 4) *Scenario IV*: As a final scenario, the behavior of different frameworks in terms of *network utilization/cost* metrics will be analyzed on the testbed.

Evaluation Metrics: Perceptual quality, algorithm performance, QoE-related, network utilization/cost metrics are defined as follows:

1) *Perceptual quality metrics*:

- *VMAF*: Video Multi-method Assessment Fusion [52] is a learning-based objective quality metric (with a range from 0 to 100) that predicts the perceived video quality.

2) *Algorithm performance metrics*:

- *ETV*: Execution Time Values (in seconds) for the different ALIVE schemes.
- *NOV*: Normalized Objective Values (*i.e.*, of the objective function Eq. (16)) for the ALIVE schemes (in %).

3) *Common QoE metrics*:

- *ASB*: Average Segment Bitrate (in Mbps) of all downloaded segments.
- *AQS*: Average Quality Switches, *i.e.*, the number of segments whose bitrate levels change compared to the previous ones.
- *ASD*: Average Stall Duration, *i.e.*, the average of total video freeze times (in seconds) in all clients.
- *ANS*: Average Number of Stalls, *i.e.*, the average number of rebuffering events.
- *ASL*: Average Serving Latency (in seconds) for serving all clients, including transmission latency plus computational latency.
- *APQ*: Average Perceived QoE (with a range from 0 to 5), calculated by ITU-T Rec. P.1203 mode 0 [69].

4) *Network utilization/cost metrics*:

- *CHR*: Cache Hit Ratio (in %), defined as the fraction of segments fetched from the CDN, VTS, or peers.
- *ETR*: Edge Transcoding Ratio (in %), *i.e.*, the fraction of segments transcoded at the VTS.
- *PTSR*: Peer SR and TR Ratio (in %), *i.e.*, the fraction of segments constructed by TR or SR at the peers.
- *BTL*: Backhaul Traffic Load (in Gbps), *i.e.*, the volume of segments downloaded from the origin server.
- *EEC*: Edge Energy Consumption (in kWh) for running ETR.
- *NCV*: Network Cost Values (in \$), including computational and bandwidth costs of the CDN network, origin, and edge servers.

We discuss details of the scenarios and ALIVE performance evaluation in the next section, where each experiment is executed 20 times to ensure accuracy, and the average values and standard deviations are reported in the experimental results.

D. Evaluation Results and Analysis

1) *Scenario I*: Running transcoding or super-resolution on peers (*i.e.*, PTR or PSR) must be sufficiently fast, not significantly imposing an extra delay to the ALIVE system, and not considerably consuming peers' batteries; otherwise, the clients' requests may use other actions that make the network

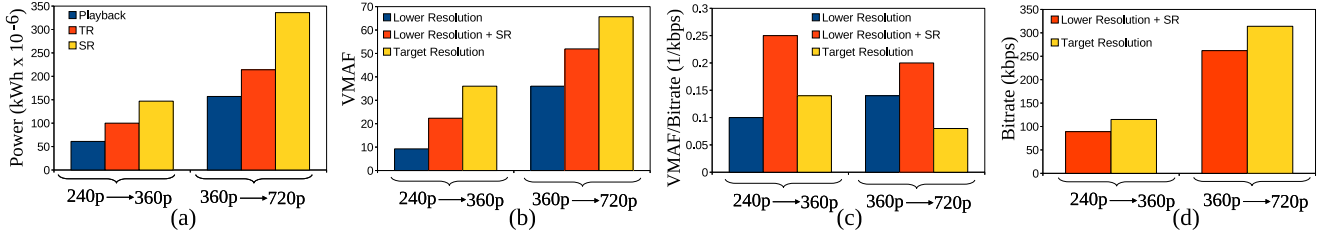


Fig. 7. Performance comparison of applying peers' SR on a lower representation in terms of (a) power consumption, (b) VMAF score, (c) VMAF/Bitrate, and (d) bandwidth saving.

and edge server congested. Hence, in the first scenario, we conduct experiments to investigate the latency, quality distortion, and energy overheads of running transcoding and super-resolution tasks on peers.

The first experiment of this scenario measures the client's power consumption when the client utilizes the PSR function to jump to a higher representation level in the bitrate ladder from the lower received representation (Fig. 7(a)). It is noteworthy that ALIVE only operates the SR on the targeted client as the last step of playing out the video. This is because the output of the SR model is a decoded representation, and then the client can directly display it. Therefore, the SR process is accomplished without inducing any extra delay, except the SR processing delay, for preparing the requested representation. However, applying SR at the edge or other network layers would require a re-encoding phase before transmitting it to the client, consequently imposing an extra encoding delay on the system. The results of this experiment are obtained for two upscaling operations on the client PC peers by the discussed CARN-M SR model. The representations used in this experiments are (89 kbps - 240p) $\rightarrow$ (262 kbps - 360p) and (262 kbps - 360p) $\rightarrow$ (791 kbps - 720p). The power consumption of peers, when they (i) play the low representation video, (ii) transcode the video from a higher representation, or (iii) apply SR from a low resolution, is plotted in Fig. 7(a). As obvious from the results, the SR application consumes much more battery than playing out the lower representation directly (about 52%), while this rise is about 38% when transcoding the requested representation from the highest representation (i.e., (4.2 Mbps - 1080p) $\rightarrow$ (262 kbps - 360p) or (791 kbps - 720p)).

We run further experiments to calculate the power consumption (i.e., power values are given as kWh $\times 10^{-3}$ and percentage of battery usage) for a five-minute video (i.e., 150 segments) when different actions are executed simultaneously on one peer (e.g., play video I, transcode video II, upscale video III, simultaneously). As reported in Table IV, the SR operation continues to use the most power, followed by the transcoding (TR) function, while battery consumption of playing back a video (i.e., PLY) is considerably lower. Moreover, a combination of the PLY, TR, and SR tasks does not put a significant burden on the peers' batteries (i.e., 4.04% and 7.74%) compared to the energy used to upscale a video individually.

The second experiment examines the quality of the video upscaled by SR in terms of the VMAF metric. As illustrated in

TABLE IV
POWER CONSUMPTION AND BATTERY USAGE OF DIFFERENT APPROACHES FOR A 5-MIN. VIDEO ON PEERS

Method	240p $\rightarrow$ 360p		360p $\rightarrow$ 720p	
	Power (kWh $\times 10^{-3}$)	Battery	Power (kWh $\times 10^{-3}$)	Battery
PLY	119	0.39%	152	0.49%
TR	130	0.42%	352	1.14%
SR	955	3.09%	1,862	6.03%
TR + PLY	237	0.77%	479	1.55%
SR + PLY	1,020	3.30%	1,915	6.20%
SR + TR	1,043	3.71%	2,036	7.25%
SR + TR + PLY	1,084	4.04%	2,079	7.74%

Fig. 7(b), by applying the peer SR function, we significantly increase the VMAF score of the displayed video compared to the lower representation (by at least 41%). Although the requested target representation still outperforms the VMAF score obtained by SR (Fig. 7(b)), applying SR provides the best VMAF score per kbps of data delivered to the client (Fig. 7(c)). We repeat the first and second experiments for different representations in the bitrate ladder to find a sweet spot in the trade-off between quality improvement and power consumption. The repeated experiments show that the sweet spot is to go up one step in the bitrate ladder, as increasing resolution too much would result in significant distortion. Therefore, upscaling the targeted representation from the closest one would result in a pleasant user's QoE (in terms of the VMAF score) and less power consumption.

In the third experiment, we investigate a solution to discover the exact bandwidth saving that can be achieved by applying SR. Assume we use the $Rep_1 = (89 \text{ kbps} - 240 \text{ p})$ representation to build $Rep_2 = (262 \text{ kbps} - 360 \text{ p})$ by SR. We first apply SR on Rep_1 to increase the resolution to Rep_2 . We then calculate the VMAF score of Rep_2 . In the next step, we encode the input source video with the requested resolution (i.e., 360p) in different bitrates (starting from the lower bitrate, i.e., 89 kbps, and gradually increasing it) to find the bitrate in which the VMAF score matches Rep_2 . We have applied the same approach to all representations in the bitrate ladder and found that applying SR can save around 24.53% bandwidth on average while maintaining the same visual quality (Fig. 7(d)).

In the final experiment of this scenario, we measure the peers' transcoding times and VMAF scores for a three-minute video. We note that the VMAF score differences in mobile and PC peers are due to using different transcoding presets, i.e., *veryfast* and *fast*, respectively. It is worth mentioning that transcoding usually needs decoding a video into raw frames

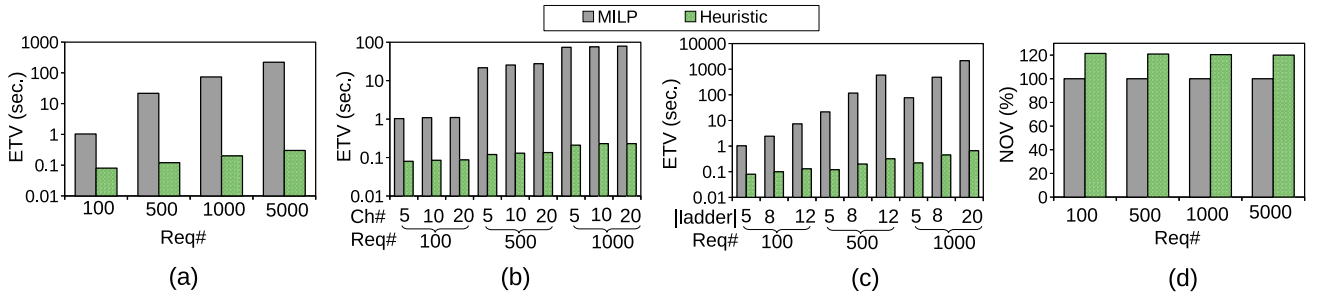


Fig. 8. Impact of (a) increasing the number of arriving requests ($Req\#$), (b) increasing the number of live channels ($Ch\#$), and (c) changing the bitrate ladder, on the performance of the MILP model and the proposed heuristic method in terms of ETV and (d) NOV.

TABLE V
TRANSCODING TIMES AND VMAF SCORES FOR 3-MIN.
VIDEO ON DIFFERENT PEER TYPES

Input BR (bps)	Target BR (bps)	Client PC		Client Mobile	
		Time (s)	VMAF	Time (s)	VMAF
4219k	89k	4.31	15.38	15.98	13.75
4219k	262k	5.33	44.61	18.32	42.13
4219k	791k	11.74	76.21	39.28	73.14
4219k	2484k	20.44	93.33	74.91	91.53
2484k	89k	3.80	14.35	16.55	13.01
2484k	262k	4.83	42.27	18.82	40.02
2484k	791k	11.36	71.56	39.76	69.06
791k	89k	2.05	12.21	10.43	11.24
791k	262k	3.35	36.33	14.81	34.76
262k	89k	1.28	11.01	5.85	10.32

and then re-encoding those frames into new frames. As shown in Table V, running transcoding for the whole video takes 1.28–20.44 seconds (0.014–0.22 seconds per segment) and 5.85–74.91 seconds (0.065–0.8 seconds per segment) on PC and mobile peers, respectively. Moreover, the results in the VMAF column indicate that using the closest input bitrate to the target bitrate achieves better VMAF scores.

2) *Scenario II*: In the second scenario, we compare the performance of the proposed MILP optimization model with the GBA-based heuristic method in terms of the algorithm performance metrics (Fig. 8).

In the first experiment of this scenario, we measure the ETV metric by (i) increasing the number of peers' requests ($Req\#$), (ii) increasing the number of live channels ($Ch\#$), and (iii) increasing the number of representations in the bitrate ladder ($|ladder|$) to assess ALIVE's GBA scalability and practicality. Fig. 8(a) highlights the ETV comparison of the ALIVE GBA scheme with the MILP model scheme when the VTS must handle different numbers of simultaneous requests (*i.e.*, $Req\# = 100, 500, 1000$, and 5000) received from peers. It should be noted that all experimental settings, including the number of other components, are the same as discussed in Section V-B for this experiment. The results show that, by increasing $Req\#$, the ETV value of the MILP model increases exponentially, while the proposed lightweight GBA achieves much less ETV.

In the second and third experiments, we investigate the impact of increasing the number of live channels (*i.e.*, $Ch\#$) and the bitrate ladder on the values of ETV, respectively. As depicted in Fig. 8(b), increasing the number of live channels

(*i.e.*, $Ch\# = 5, 10, 20$) does not have a significant effect on the ETV values and slightly increases it in both the MILP and heuristic methods. In the next experiment, we examine the behavior of the proposed schemes when using a different number of representations in the bitrate ladders (*i.e.*, $|ladder| = 5, 8, 12$). As plotted in Fig. 8(c), replacing a bitrate ladder with a ladder with more representations increases the ETV values noticeably. In fact, having a bitrate ladder with more representations boosts the problem search space, consequently causing schemes to spend much more time deciding on the best nodes and actions. It is evident in all experiments that the proposed GBA-based heuristic scheme outperforms the MILP model significantly in terms of the ETV, showing modest growth with changing the above-mentioned parameters (see Fig's. 8(a–c)).

In the last experiment of this scenario, we measure the normalized objective function value (*i.e.*, NOV) metric by varying the number of requests ($Req\#$). As displayed in Fig. 8(d), the MILP model surpasses the GBA method regarding the NOV value for all $Req\#$. We note that having more live channels and/or replacing the bitrate ladder yield similar NOV results. As described earlier, the MILP model is executed for all peers' requests, while the GBA method is run based on regions' queues in a time-slotted fashion. The results of this experiment confirm that the GBA-based heuristic results are still close to the MILP model (Fig. 8(d)).

3) *Scenario III*: This scenario investigates the performance of the proposed ALIVE GBA method on the introduced testbed and compares the common QoE metrics with baseline methods. This scenario does not use the ALIVE MILP method due to its high time complexity and impracticality in large-scale environments.

As illustrated in Fig. 9(a), ALIVE serves HAS clients with higher ASB in both BOLA and SQUAD-based systems. This is because ALIVE uses all computational, caching, and bandwidth resources in a hybrid P2P-CDN network. Similar to ASB, Fig. 9(b) demonstrates that ALIVE decreases the AQS for both ABR algorithms by at least 34.5%. This is again due to using all provided resources in the ALIVE system for fetching, running TR, or operating SR compared to other methods.

The effectiveness of different systems in terms of ANS and ASD, as essential factors of users' QoE, are shown in Fig's. 9(c) and (d), respectively. It is worth noting that

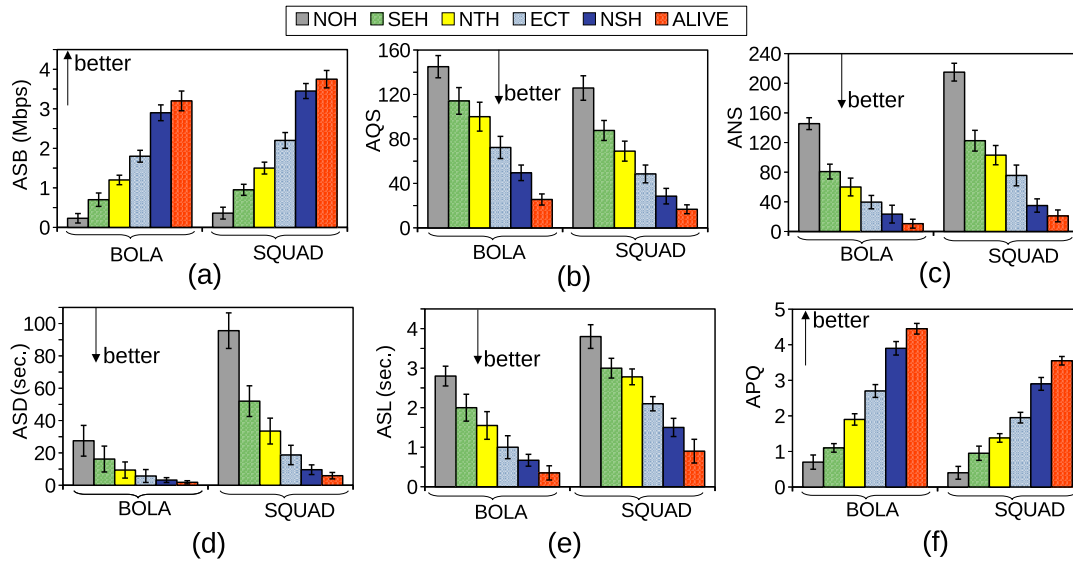


Fig. 9. Performance of the ALIVE system compared with baseline methods in terms of (a) ASB, (b) AQS, (c) ANS, (d) ASD, (e) ASL, and (f) APQ for 350 clients running the BOLA and SQUAD ABR algorithms.

buffer-based ABR algorithms (*e.g.*, BOLA) are focused on the buffer status for adaptation decisions; thus, they work better than SQUAD-based players in terms of the ANS and ASD metrics. Utilizing the GBA algorithm as a decision-maker for selecting the best node and best action with minimum delay, enables the ALIVE system to serve HAS requests with fewer ANS and ASD values. To be precise, ALIVE decreases ANS by at least 40% and shortens ASD for BOLA by at least 43% compared to other systems. Results for the SQUAD-based systems follow the same trend, where ANS and ASD values obtained by the ALIVE system outperform baseline approaches by at least 42% and 45%, respectively. Moreover, we measure the ASL, which is considered an essential part of the end-to-end delay. As illustrated in Fig. 9(e), the ALIVE system demonstrates superior performance for both ABR algorithms in terms of the ASL metric. In fact, downloading requested segments via the most appropriate actions with minimum serving time (computation plus transmission) and from the best node with high available resources (*i.e.*, bandwidth, computation) significantly shortens both ASD and ASL values, where this improvement for ASL is at least 40% compared to other methods. It is noteworthy that since SQUAD-based players download higher-quality segments and their ASD values are longer than those of BOLA-based players (Fig. 9(a) and (d)), their ASL values are higher than those of BOLA-based players.

Although enhancing each QoE parameter individually (*e.g.*, ASB, AQS, ANS, ASD, and ASL) can generally improve the users' satisfaction, having a comprehensive standard model is essential to analyze the system behavior in terms of users' QoE improvement. Therefore, we use a standard QoE model [69] to evaluate ALIVE's performance in terms of the APQ metric. As plotted in Fig. 9(f), the value of the APQ metric is between zero and five, where zero is the worst-perceived quality and five is the best-perceived quality. It is worth mentioning that stall duration (measured by the ASD metric) significantly

affects APQ compared to other common QoE parameters. Hence, BOLA-based players with better ASD values achieve better APQ compared to SQUAD-based players. As shown in Fig. 9(f), the ALIVE system presents superior performance for both ABR algorithms. In fact, ALIVE enhances APQ by at least 22% compared to other baseline techniques. Improving the ASB, AQS, ANS, and particularly shortening the ASD values, is the main reason for outperforming the NSH system as the second-best scheme in terms of the APQ metric.

4) *Scenario IV*: In the fourth scenario, we analyze the performance of the proposed ALIVE GBA scheme on the testbed regarding the network utilization/cost utilization metrics and compare the obtained results with baseline methods. Similar to scenario III, we do not utilize the ALIVE MILP model in this scenario.

The performance analysis of the ALIVE scheme concerning the CHR and ETR metrics are depicted in Fig's. 10(a) and (b), respectively. It should be noted that different frameworks encounter cache miss events under various conditions, which force them to fetch the requested representations from the origin servers. These conditions are listed as follows:

- *Condition I*: The requested representations are not held in CDN servers (*e.g.*, for NOH).
- *Condition II*: The requested representations do not exist in CDN servers or peers (*e.g.*, for SEH).
- *Condition III*: The requested representations are not in CDN servers, VTS, or neighboring peers (*e.g.*, for NTH).
- *Condition IV*: Available bandwidth is insufficient to fetch the requested representation from the selected server (*i.e.*, CDN or VTS) or peer (*e.g.*, for NOH, SEH, or NTH).
- *Condition V*: The requested or higher representations do not exist in CDN servers; the VTS server does not hold the requested or higher representations; the VTS's resources are insufficient for serving requests via the ETR function. In addition, the requested representations are not cached in any adjacent peer (*e.g.*, for ECT).

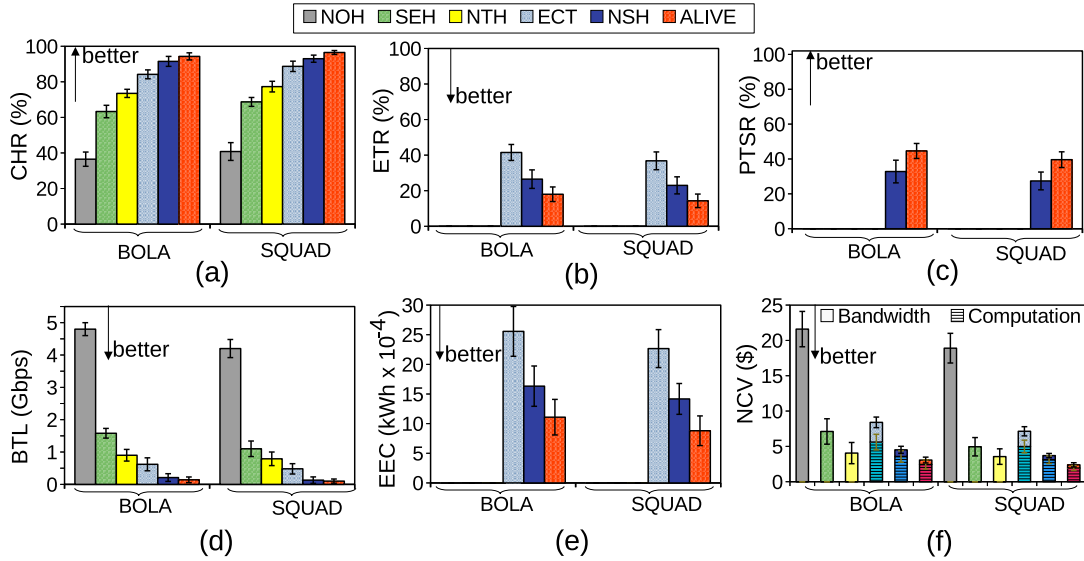


Fig. 10. Performance of the ALIVE system compared with baseline methods in terms of (a) CHR, (b) ETR, (c) PTSR, (d) BTL, (e) EEC, and (f) NCV for 350 clients running the BOLA and SQUAD ABR algorithms.

- *Condition VI:* Peers do not have the candidate representations (*i.e.*, higher or lower) for serving requests through the PTR or PSR functions; peers' resources (*i.e.*, computational or bandwidth) are inadequate for serving requests by the PTR or PSR operations (*e.g.*, for NSH and ALIVE).

Under such circumstances, ALIVE's VTS server acts as a proxy, fetching the requested representations from the origin server and forwarding them to the clients. The CHR metric plotted in Fig. 10(a) shows that ALIVE outperforms the NOH, SEH, NTH, and ECT approaches by at least 12% due to its ability to fetch requested, higher, or lower representations in a hybrid system and then to reconstruct the requested representations via the PRT, PSR, or ETR functions. Moreover, ALIVE slightly improves over the NSH system regarding the CHR (by 4%) due to having an extra action 3 for fetching the lower representation from adjacent peers. As shown in Fig. 10(b), ALIVE improves the ETR metric of the ECT system by 53% due to using idle computational resources of the P2P network for running the PTR and PSR functions. Note that since NOH, SEH, and NTH are not transcoding-enabled systems, the ETR metrics does not apply. Moreover, since ALIVE employs the PSR function besides PTR, it improves over the NSH system by 32% in terms of ETR.

The PTSR metric in Fig. 10(c) indicates that ALIVE increases the usage of available computational resources of the P2P network (by 36%) compared to the NSH framework, which is the only baseline approach that employs peers' computations for running PTR. This is because ALIVE operates super-resolution by the PSR function besides direct fetching and transcoding for serving requests. However, improving the cache hit ratio (*i.e.*, CHR) and constructing requested representations at the edge or P2P network (*i.e.*, by ETR, PTR, or PSR) directly affects the BTL metric since the system downloads fewer segments from the origin server; ALIVE thus outperforms other systems and significantly reduces fetching

representations from the origin server, by at least 24%, compared to other frameworks (Fig. 10(d)).

The next experiment analyzes the EEC metric to assess edge server energy consumption for producing requested representations in different systems. As is evident from Fig. 10(e), ALIVE has better EEC results than the NSH and ECT methods (the only transcoding-enabled baseline systems) and reduces energy consumption by at least 31%. Indeed, serving requested representations through (i) the representations cached in all layers of the network (CDN, P2P, and edge), (ii) the representations transcoded by transcoding-enabled peers, or (iii) the resolutions up-scaled from the lower representations with smaller sizes at the peer, allows ALIVE to transcode fewer segments at the edge, consequently consuming less energy. In the final experiment, we measure the incurred costs to providers, based on network bandwidth and computational costs, in different systems. As plotted in Fig. 10(f), the ALIVE scheme achieves the best performance among all the methods and reduces network costs by at least 34%. Employing all resources provided by peers improves the CHR and ETR metrics and decreases BTL as a network cost factor. Note that increasing the number of segments held by peers may result in fetching more segments directly from those peers rather than utilizing top layers or executing PTR and PSR.

VI. CONCLUSION

This article presented ALIVE, a latency- and cost-aware hybrid P2P-CDN architecture for HAS-based live video streaming applications. We (i) leveraged modern networking paradigms (*e.g.*, NFV, edge computing), distributed video transcoding, and video super-resolution techniques to introduce a multi-layer architecture and an action tree that defines all possible actions to serve HAS clients (from peers, CDN, edge, or origin servers) with satisfactory users' QoE and acceptable latency, (ii) formulated the problem as a

Mixed-Integer Linear Programming (MILP) model subject to the different network resource constraints, and (iii) solved the problem using a lightweight heuristic solution. For the performance evaluation, a large-scale cloud-based testbed consisting of 350 clients was designed and instantiated, and then ALIVE's effectiveness was investigated on it through four comprehensive scenarios. Experimental results demonstrated the superiority of ALIVE in terms of users' QoE (by at least 22%), serving latency (by at least 40%), edge server's energy consumption (by at least 31%), network cost (by at least 34%), and backhaul bandwidth usage (by at least 24%), compared to competitive schemes. Our future work aims to design a sustainable and energy-conscious hybrid P2P-CDN video delivery approach.

REFERENCES

- [1] "The global Internet phenomena report," Sandvine, Waterloo, ON, Canada, White Paper, Jan. 2023. [Online]. Available: <https://www.sandvine.com/phenomena>
- [2] I. Sodagar, "The MPEG-DASH standard for multimedia streaming over the Internet," *IEEE Multimedia*, vol. 18, no. 4, pp. 62–67, Apr. 2011.
- [3] R. Pantos and W. May, "HTTP live streaming," Internet Eng. Task Force, Fremont, CA, USA, RFC 8216, Aug. 2017. [Online]. Available: <https://rfc-editor.org/rfc/rfc8216.txt>
- [4] A. Bentaleb, B. Taani, A. C. Begen, C. Timmerer, and R. Zimmermann, "A survey on bitrate adaptation schemes for streaming media over HTTP," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 1, pp. 562–585, 1st Quart., 2019.
- [5] N.-N. Dao, A.-T. Tran, N. H. Tu, T. T. Thanh, V. N. Q. Bao, and S. Cho, "A contemporary survey on live video streaming from a computation-driven perspective," *ACM Comput. Surveys*, vol. 54, p. 202, Nov. 2022.
- [6] A. A. Barakabitze et al., "QoE management of multimedia streaming services in future networks: A tutorial and survey," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 1, pp. 526–565, 1st Quart., 2020.
- [7] D. Bhat, A. Rizk, M. Zink, and R. Steinmetz, "SABR: Network-assisted content distribution for QoE-driven ABR video streaming," *ACM Trans. Multimedia Comput. Commun. Appl.*, vol. 14, no. 2s, p. 32, 2018.
- [8] S. Bayhan, S. Maghsudi, and A. Zubow, "EdgeDASH: Exploiting network-assisted adaptive video streaming for edge caching," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 2, pp. 1732–1745, Jun. 2021.
- [9] R. Farahani, F. Tashtarian, A. Erfanian, C. Timmerer, M. Ghanbari, and H. Hellwagner, "ES-HAS: An edge- and SDN-assisted framework for HTTP adaptive video streaming," in *Proc. 31st ACM NOSSDAV Workshop*, 2021, pp. 50–57.
- [10] R. Farahani, F. Tashtarian, H. Amirpour, C. Timmerer, M. Ghanbari, and H. Hellwagner, "CSDN: CDN-aware QoE optimization in SDN-assisted HTTP adaptive video streaming," in *Proc. 46th Conf. IEEE Local Comput. Netw. (LCN)*, 2021, pp. 525–532.
- [11] C. Li, L. Zhu, W. Li, and Y. Luo, "Joint edge caching and dynamic service migration in SDN based mobile edge computing," *J. Netw. Comput. Appl.*, vol. 177, Mar. 2021, Art. no. 102966.
- [12] R. Farahani, F. Tashtarian, C. Timmerer, M. Ghanbari, and H. Hellwagner, "LEADER: A collaborative edge- and SDN-assisted framework for HTTP adaptive video streaming," in *Proc. IEEE Int. Conf. Commun.*, 2022, pp. 745–750.
- [13] R. Farahani, "CDN and SDN support and player interaction for HTTP adaptive video streaming," in *Proc. 12th ACM Multimedia Syst. Conf.*, 2021, pp. 398–402.
- [14] S. Nacakli and A. M. Tekalp, "Controlling P2P-CDN live streaming services at SDN-enabled multi-access edge datacenters," *IEEE Trans. Multimedia*, vol. 23, pp. 3805–3816, 2020.
- [15] S. Budhkar and V. Tamarapalli, "An overlay management strategy to improve QoS in CDN-P2P live streaming systems," *Peer-to-Peer Netw. Appl.*, vol. 13, pp. 190–206, Jan. 2020.
- [16] M. Chiosi et al., "Network functions virtualisation: An introduction, benefits, enablers, challenges and call for action," in *Proc. SDN OpenFlow World Congr.*, vol. 48, 2012, pp. 22–24.
- [17] H. A. Pedersen and S. Dey, "Enhancing mobile video capacity and quality using rate adaptation, RAN caching and processing," *IEEE/ACM Trans. Netw.*, vol. 24, no. 2, pp. 996–1010, Apr. 2016.
- [18] X. Zhang et al., "Optimizing video caching at the edge: A hybrid multi-point process approach," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 10, pp. 2597–2611, Oct. 2022.
- [19] G. Ma, Z. Wang, M. Zhang, J. Ye, M. Chen, and W. Zhu, "Understanding performance of edge content caching for mobile video streaming," *IEEE J. Sel. Areas Commun.*, vol. 35, no. 5, pp. 1076–1089, May 2017.
- [20] R. Farahani, M. Shojafar, C. Timmerer, F. Tashtarian, M. Ghanbari, and H. Hellwagner, "ARARAT: A collaborative edge-assisted framework for HTTP adaptive video streaming," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 1, pp. 625–643, Mar. 2023.
- [21] A. Zhang et al., "Video super-resolution and caching—An edge-assisted adaptive video streaming solution," *IEEE Trans. Broadcast.*, vol. 67, no. 4, pp. 799–812, Dec. 2021.
- [22] M. Zhao et al., "Peer-assisted content distribution in Akamai NetSession," in *Proc. Conf. Internet Meas. Conf.*, 2013, pp. 31–42.
- [23] Z. Ma, S. Roubia, F. Giroire, and G. Urvoy-Keller, "When locality is not enough: Boosting peer selection of hybrid CDN-P2P live streaming systems using machine learning," in *Proc. Netw. Traffic Meas. Anal. Conf. (IFIP TMA)*, 2021, pp. 1–9.
- [24] T. T. T. Ha, J. Kim, and J. Nam, "Design and deployment of low-delay hybrid CDN-P2P architecture for live video streaming over the Web," *Wireless Pers. Commun.*, vol. 94, no. 3, pp. 513–525, 2017.
- [25] E. Baccour, A. Erbad, A. Mohamed, M. Guizani, and M. Hamdi, "Collaborative hierarchical caching and transcoding in edge network with CE-D2D communication," *J. Netw. Comput. Appl.*, vol. 172, Dec. 2020, Art. no. 102801.
- [26] H. Yousef, J. Le Feuvre, P.-L. Ageneau, and A. Storelli, "Enabling adaptive bitrate algorithms in hybrid CDN/P2P networks," in *Proc. 11th ACM Multimedia Syst. Conf.*, 2020, pp. 54–65.
- [27] R. Farahani, A. Bentaleb, E. Çetinkaya, C. Timmerer, R. Zimmermann, and H. Hellwagner, "Hybrid P2P-CDN architecture for live video streaming: An online learning approach," in *Proc. IEEE Glob. Commun. Conf. (GLOBECOM)*, 2022, pp. 1911–1917.
- [28] M. Choi, J. Kim, and J. Moon, "Wireless video caching and dynamic streaming under differentiated quality requirements," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 6, pp. 1245–1257, Jun. 2018.
- [29] R. Farahani, A. Bentaleb, C. Timmerer, M. Shojafar, R. Prodan, and H. Hellwagner, "SARENA: SFC-enabled architecture for adaptive video streaming applications," in *Proc. IEEE Int. Conf. Commun.*, 2023, pp. 864–870.
- [30] E. Thomas, M. van Deventer, T. Stockhammer, A. C. Begen, M.-L. Champel, and O. Oyman, "Applications and deployments of server and network assisted DASH (SAND)," in *Proc. IBC Conf.*, 2016, pp. 1–8. [Online]. Available: <https://digital-library.theiet.org/content/conferences/10.1049/ibc.2016.0022>
- [31] "Web application video ecosystem—common media client data," CTA, Hopewell, VA, USA, CTA-5004, 2020. [Online]. Available: <https://cdn.cta.tech/cta/media/media/resources/standards/pdfs/cta-5004-final.pdf>
- [32] (CTA, Hopewell, VA, USA). "Common-media-server data, CTA-WAVE." 2021. [Online]. Available: <https://github.com/cta-wave/common-media-client-data/issues/19>
- [33] N. Anjum, D. Karamshuk, M. Shikh-Bahaei, and N. Sastry, "Survey on peer-assisted content delivery networks," *Comput. Netw.*, vol. 116, pp. 79–95, Apr. 2017.
- [34] H. Xie, Y. R. Yang, A. Krishnamurthy, Y. G. Liu, and A. Silberschatz, "P4P: Provider portal for applications," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 4, pp. 351–362, 2008.
- [35] R. Alimi et al., "Application-layer traffic optimization (ALTO) protocol," IETF, Fremont, CA, USA, RFC 7285, 2014.
- [36] S. Ellouze, B. Mathieu, and T. Lemlouma, "A bidirectional network collaboration interface for CDNs and clouds services traffic optimization," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2013, pp. 3592–3596.
- [37] M. Thornburgh, "Adobe's secure real-time media flow protocol," IETF, Fremont, CA, USA, RFC 7016, 2013. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7016.html>
- [38] S. Zou, Q. Wang, J. Ge, and Y. Tian, "Peer-assisted video streaming with RTMP flash player: A measurement study on PPTV," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 28, no. 1, pp. 158–170, Jan. 2018.
- [39] R. Farahani et al., "RICHTER: Hybrid P2P-CDN architecture for low latency live video streaming," in *Proc. ACM Mile-High Video (MHV)*, 2022, pp. 87–88.
- [40] E. Borcoci, D. Negru, and C. Timmerer, "A novel architecture for multimedia distribution based on content-aware networking," in *Proc. 3rd Int. Conf. Commun. Theory Reliabil. Qual. Service*, 2010, pp. 162–168.

- [41] M. Conforti, G. Cornuéjols, and G. Zambelli, *Integer Programming*, vol. 271. Cham, Switzerland: Springer, 2014.
- [42] M. R. Garey and D. S. Johnson, *Computers and Intractability. A Guide to the Theory of NP-Completeness*. San Francisco, CA, USA: W. H. Freeman and Company, 1979.
- [43] Z. Wang, J. Chen, and S. C. Hoi, "Deep learning for image super-resolution: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 10, pp. 3365–3387, Oct. 2021.
- [44] C. Dong, C. C. Loy, K. He, and X. Tang, "Image super-resolution using deep convolutional networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 38, no. 2, pp. 295–307, Feb. 2016.
- [45] W. Shi et al., "Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 1–10.
- [46] E. Çetinkaya, M. Nguyen, and C. Timmerer, "LiDeR: Lightweight dense residual network for video super-resolution on mobile devices," in *Proc. IEEE 14th Image Video Multidimensional Signal Process. Workshop (IVMSP)*, 2022, pp. 1–5.
- [47] C. Dong, C. C. Loy, and X. Tang, "Accelerating the super-resolution convolutional neural network," in *Proc. Eur. Conf. Comput. Vis.*, 2016, pp. 391–407.
- [48] B. Lim, S. Son, H. Kim, S. Nah, and K. Mu Lee, "Enhanced deep residual networks for single image super-resolution," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops*, 2017, pp. 1–9.
- [49] N. Ahn, B. Kang, and K.-A. Sohn, "Fast, accurate, and lightweight super-resolution with cascading residual network," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 256–272.
- [50] G. Gouvine, "NinaSR: Efficient small and large ConvNets for super-resolution." 2021. [Online]. Available: <https://github.com/Coloquinte/torchSR/blob/main/doc/NinaSR.md>
- [51] Y. Zhang, K. Li, K. Li, L. Wang, B. Zhong, and Y. Fu, "Image super-resolution using very deep residual channel attention networks," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 294–310.
- [52] Z. Li, A. Aaron, I. Katsavounidis, A. Moorthy, and M. Manohara, "Toward a practical perceptual video quality metric." 2016. Accessed: Dec. 25, 2022. [Online]. Available: <https://netflixtechblog.com/toward-a-practical-perceptual-video-quality-metric-653f208b9652>
- [53] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 1–6.
- [54] Accessed: Dec. 25, 2022. [Online]. Available: <http://www.topology-zoo.org/dataset.html>
- [55] R. Ricci, E. Eide, and C. Team, "Introducing cloudLab: Scientific infrastructure for advancing cloud architectures and applications," *Mag. USENIX & SAGE*, vol. 39, no. 6, pp. 36–38, 2014.
- [56] P. Juluri, V. Tamarapalli, and D. Medhi, "SARA: Segment aware rate adaptation algorithm for dynamic adaptive streaming over HTTP," in *Proc. IEEE Int. Conf. Commun. Workshops (ICCW)*, 2015, pp. 1765–1770.
- [57] C. Wang, A. Rizk, and M. Zink, "SQUAD: A spectrum-based quality adaptation for dynamic adaptive streaming over HTTP," in *Proc. 7th Int. Conf. Multimedia Syst.*, 2016, pp. 1–12.
- [58] K. Spiteri, R. Ugaonkar, and R. K. Sitaraman, "BOLA: Near-optimal bitrate adaptation for online videos," in *Proc. 35th Annu. IEEE Int. Conf. Comput. Commun.*, 2016, pp. 1–9.
- [59] S. Lederer, C. Müller, and C. Timmerer, "Dynamic adaptive streaming over HTTP dataset," in *Proc. 3rd ACM Multimedia Syst. Conf.*, 2012, pp. 89–94.
- [60] L. Cherkasova and M. Gupta, "Analysis of enterprise media server workloads: access patterns, locality, content evolution, and rates of change," *IEEE/ACM Trans. Netw.*, vol. 12, no. 5, pp. 781–794, Oct. 2004.
- [61] Accessed: Dec. 25, 2022. [Online]. Available: <https://hub.docker.com/r/jrottenberg/ffmpeg>
- [62] Accessed: Dec. 25, 2022. [Online]. Available: <https://github.com/athena/ffmpeg-kit>
- [63] V. Schmidt et al., "CodeCarbon: Estimate and track carbon emissions from machine learning computing." 2021. [Online]. Available: <https://github.com/mlco2/codecarbon>
- [64] S. Yi, Z. Hao, Q. Zhang, Q. Zhang, W. Shi, and Q. Li, "LAVEA: Latency-aware video analytics on edge computing platform," in *Proc. 2nd ACM/IEEE Symp. Edge Comput.*, 2017, p. 15.
- [65] A. O. Al-Abbasi, V. Aggarwal, and M.-R. Ra, "Multi-tier caching analysis in CDN-based over-the-top video streaming systems," *IEEE/ACM Trans. Netw.*, vol. 27, no. 2, pp. 835–847, Apr. 2019.
- [66] Accessed: Dec. 25, 2022. <https://github.com/magnifico/wondershaper>
- [67] D. Raca, J. J. Quinlan, A. H. Zahran, and C. J. Sreenan, "Beyond throughput: A 4G LTE dataset with channel and context metrics," in *Proc. 9th ACM Multimedia Syst. Conf.*, 2018, pp. 460–465.
- [68] Accessed: Dec. 25, 2022. [Online]. Available: <https://calculator.aws/>
- [69] "Parametric bitstream-based quality assessment of progressive download and adaptive audiovisual streaming services over reliable transport - video quality estimation module," ITU-T, Geneva, Switzerland, 2017, p. 1203. [Online]. Available: <http://handle.itu.int/11.1002/ps/P1203-01>



Reza Farahani (Member, IEEE) received the M.Sc. degree in information technology and computer engineering from the University of Klagenfurt, Klagenfurt, Austria, in 2019, and the Ph.D. degree in information technology and computer engineering from the University of Tehran, Tehran, Iran, in 2023. He is a Postdoctoral Researcher in the Horizon Europe GraphMassivizer Project with the Institute of Information Technology (ITEC), University of Klagenfurt. From October 2019 to February 2023, he was involved in the ATHENA Project, funded by the Christian Doppler Forschungsgesellschaft and the industry partner Bitmovin GmbH. From November 2022 to January 2023, he was a Visiting Scholar with the 5G&6G Innovation Centre, Institute for Communication Systems, University of Surrey, U.K. He has coauthored more than 20 publications in international conferences and journals. Additionally, he has held the role of technical program chair for several international conferences. He has also been working in the computer networks field in different roles, e.g., a network engineer, a developer, a protocol designer, and an instructor (R&S, SP) for over six years. His research interests are network and service management, parallel and distributed systems, multimedia communication, edge-cloud continuum, serverless computing, network softwareization/virtualization, mathematics optimization, and distributed learning approaches. For more information, see <https://www.rezafarahani.me/>.



Ekrem Çetinkaya (Member, IEEE) received the B.Sc. and M.Sc. degrees in computer science from Ozyegin University, Istanbul, Türkiye, in 2018 and 2019, respectively, and the Ph.D. degree from Alpen-Adria-Universität, Klagenfurt, Austria, in 2023. He worked on the content provisioning part of the ATHENA Project, and he contributed to several publications and patent applications. His current research interests include video encoding enhancements with machine learning. For more information, see <https://ekrcet.com/>.



Christian Timmerer (Senior Member, IEEE) is a Full Professor of Computer Science with Institute of Information Technology (ITEC), Alpen-Adria-Universität Klagenfurt, and he is the Director of Christian Doppler Laboratory ATHENA. His research interests include multimedia systems, immersive multimedia communication, streaming, adaptation, and quality of experience, where he coauthored eight patents and more than 300 articles. He was the General Chair of WIAMIS 2008, QoMEX 2013, MMSys 2016, and PV 2018 and has participated in several EC-funded projects, notably DANAE, ENTHRON, P2P-Next, ALICANTE, SocialSensor, COST IC1003 QUALINET, ICoSOLE, and SPIRIT. He also participated in ISO/MPEG work for several years, notably in MPEG-21, MPEG-M, MPEG-V, and MPEG-DASH, where he served as a Standard Editor. In 2012, he cofounded Bitmovin to provide professional services around MPEG-DASH, where he holds the position of the Chief Innovation Officer—Head of Research and Standardization. For more information, see <http://timmerer.com>.



Mohammad Shojafar (Senior Member, IEEE) received the Ph.D. degree in ICT from the Sapienza University of Rome, Rome, Italy, in 2016, with an “Excellent” degree. He is a Senior Lecturer (Associate Professor) in the network security and an Intel Innovator, and a Marie Curie Alumni, working with the 5G & 6G Innovation Centre (5GIC & 6GIC), Institute for Communication Systems, University of Surrey, U.K. Before joining 5GIC & 6GIC, he was a Senior Researcher and a Marie Curie Fellow in the SPRITZ Security and Privacy

Research Group with the University of Padua, Italy. He secured around £1.2M as a PI in various EU/U.K. projects, including 5G Mode (funded by DSIT/UK; 2023), TRACE-V2X (funded by EU/MSCA-SE; 2023), D-Xpert (funded by I-UK; 2023), AUTOTRUST (funded by ESA/EU; 2021), PRISENODE (funded by EU/MSCA-IF:2019), and SDN-Sec (funded by Italian Government:2018). He was also a COI of various UK/EU projects like HiPER-RAN (funded by DSIT/UK; 2023), APTd5G Project (funded by EPSRC/UKI-FNI: 2022), ESKMARALD (funded by UK/NCSC; 2022), GAUCHO, S2C, and SAMMClouds (funded by Italian Government; 2016–2018). He is an Associate Editor of IEEE TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT, IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS, IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS FOR VIDEO TECHNOLOGY and *Computer Networks*. He is a Professional ACM Member and an ACM Distinguished Speaker, and a Fellow of the Higher Education Academy. For more information, see <http://mshojafar.com>.



Mohammad Ghanbari (Life Fellow, IEEE) is an Emeritus Professor with the School of Computer Science and Electronic Engineering, University of Essex, U.K. He is currently involved in the ATHENA Project with Alpen-Adria-Universität Klagenfurt, Austria. He is internationally best known for the pioneering work on layered video coding, which earned him IEEE Fellowship in 2001. He has registered for thirteen international patents and published more than 770 technical papers on various aspects of video networking, many of which have had

fundamental influences in this field. These include video/image compression, layered/scalable video coding, video transcoding, motion estimation, and video quality metrics. He is the author and a coauthor of eight books, and his book *Video Coding: An Introduction to Standard Codecs* (IET Press, 1999), received the Rayleigh prize as the best book of the year 2000 by IET. He was one of the founding Associate Editor of IEEE TRANSACTION ON MULTIMEDIA from 1999 to 2002.



Hermann Hellwagner (Senior Member, IEEE) is a Full Professor of Computer Science with Alpen-Adria-Universität Klagenfurt, where he leads the research group Multimedia Communication with the Institute of Information Technology (ITEC). Earlier, he held positions as an Associate Professor with the University of Technology in Munich and as a Senior Researcher with Siemens Corporate Research, Munich. He has published widely on parallel computer architecture, parallel programming, and multimedia communication and adaptation. His

current research areas are distributed multimedia systems, multimedia communication and adaptation, QoS/QoE, and communication in multi-UAV networks. He was a member of the Scientific Board of the Austrian Science Fund from 2005 to 2016 and an FWF Vice President from 2013 to 2016. He is currently a member of the CD Senate of Christian Doppler Forschungsgesellschaft. He is a Senior Member of ACM. For more information, see <https://www.itec.aau.at/~hellwagn/>.